

# Homework - 4

Repository: <https://github.com/Ani2508/data260-3209>

**Domain: Clinical Trial Listing**

**Personal Configuration**

| Value       | Result                     |
|-------------|----------------------------|
| SID4        | 3209                       |
| PORT_BASE   | 8509 (8000 + 3209 mod 900) |
| PREFIX      | s3209                      |
| SEED        | 3209                       |
| VERIFY_SEED | 263209                     |
| DOMAIN_ID   | 1 -Clinical trial listings |

**Environment**

|              |                                                                                                 |
|--------------|-------------------------------------------------------------------------------------------------|
| Hardware     | Microsoft Surface Laptop 3 — AMD Ryzen 5 (4 cores), integrated AMD Radeon (no GPU acceleration) |
| RAM          | 16GB                                                                                            |
| OS           | Windows 11 Home 64-bit                                                                          |
| Local model: | qwen3:4b-instruct-2507-q4_K_M                                                                   |

**Part1**

This part builds a React frontend for the Clinical Trial Listing app. It talks to the FastAPI backend from Part 2. A user must log in before they can see or change any trial record.

**Home Page:**

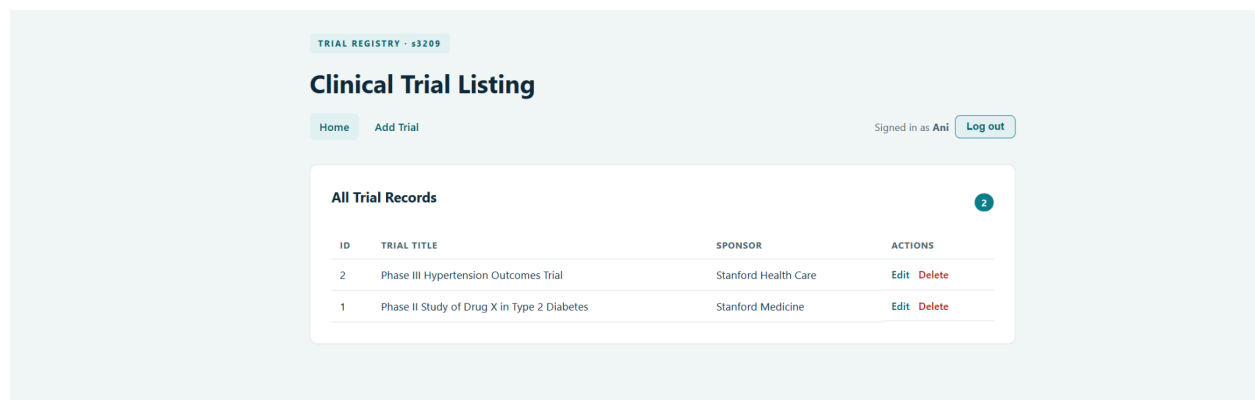
Home.jsx renders at the root route /. It shows all trial records in a table: ID, Trial Title, Sponsor, and Edit/Delete links. If the user is not logged in, it shows the "Login required".

```
export default function Home({ user, trials, loading, error }) {
  if (!user) {
    return (
      <section className="card narrow">
        <p className="state state-info">Login required</p>
        <p className="side-empty">Please <Link to="/login">log in</Link> to view and manage trial records.</p>
      </section>
    )
  }

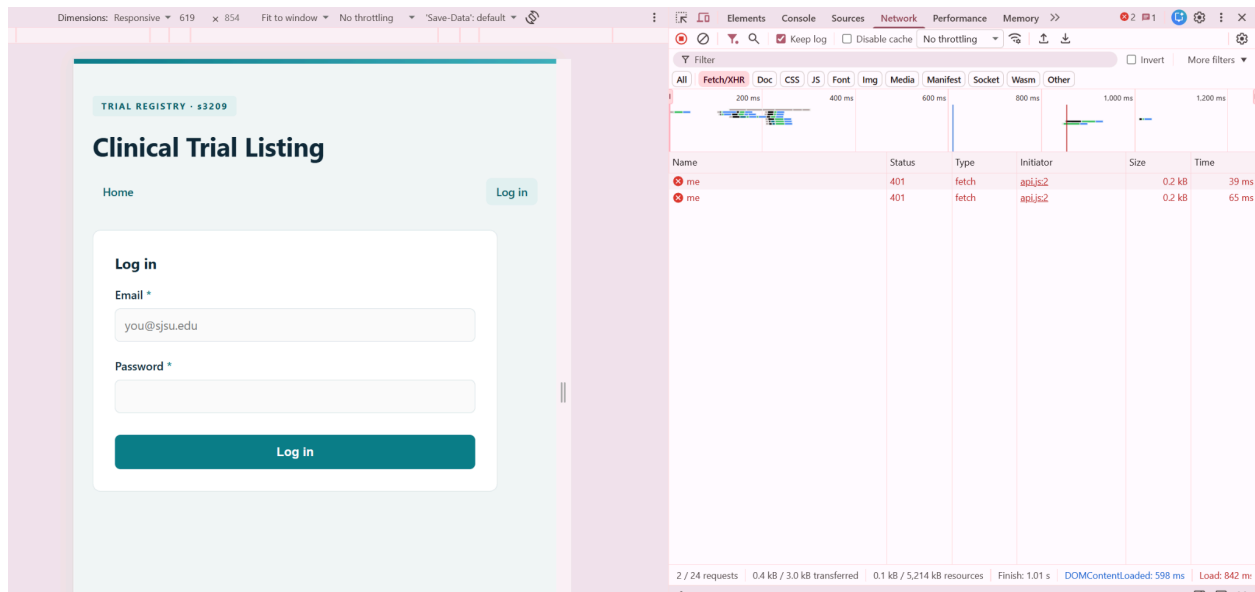
  return (
    <section className="card">
      <div className="side-head">
        <h2>All Trial Records</h2>
        <span className="count-badge">{trials.length}</span>
      </div>

      {loading && <p className="state state-loading">Loading trial records...</p>}
      {error && <p className="state state-error">{error}</p>}
      {!loading && !error && trials.length === 0 &&
        <p className="side-empty">No trials yet. Click "Add Trial" to create one.</p>}

      {trials.length > 0 && (
        <div className="table-wrap">
          <table className="trial-table">
            <thead>
              <tr>
                <th>ID</th>
                <th>Trial Title</th>
                <th>Sponsor</th>
                <th>Actions</th>
              </tr>
            </thead>
            <tbody>
              {trials.map((t) => (
                <tr key={t.id}>
                  <td>{t.id}</td>
                  <td>{t.trialTitle}</td>
                  <td>{t.sponsor}</td>
                  <td className="row-actions">
                    <Link to={` /update?id=${t.id}`}>Edit</Link>
                    <Link to={` /delete?id=${t.id}`} className="danger">Delete</Link>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </section>
  )
}
```



Reload while logged out :



## Log in

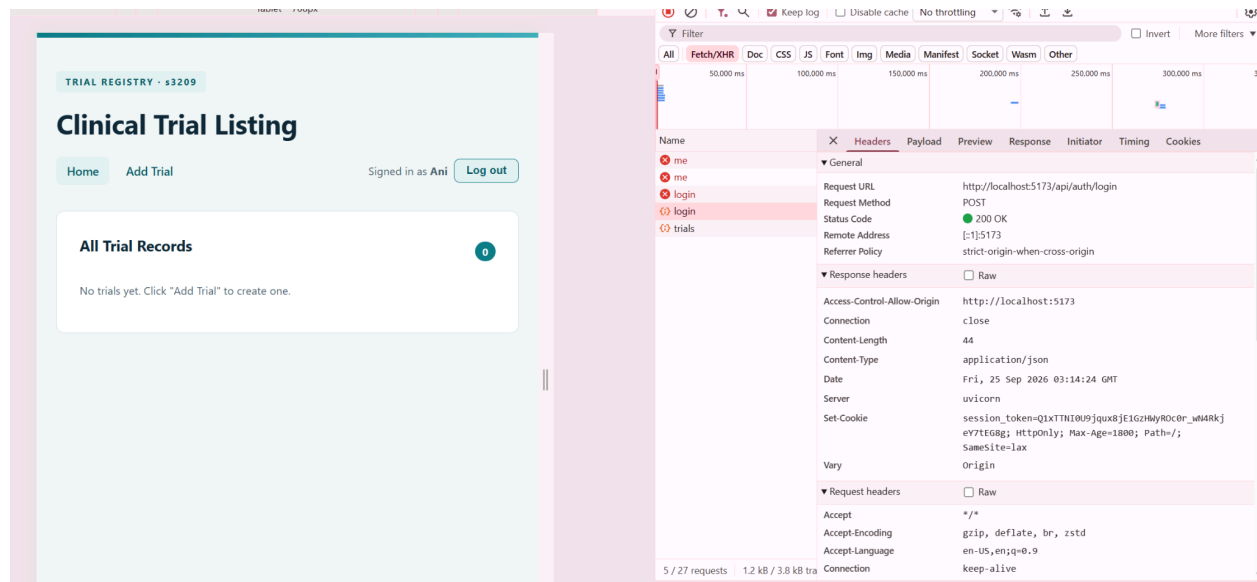
Login.jsx shows an email and password form. On submit, it calls POST /api/auth/login. If the login is correct, the backend creates a row in the sessions table and sends back a cookie named session\_token. The cookie is HttpOnly, so JavaScript in the browser cannot read it. If there is no valid session, the app shows "Login required" and hides the "Add Trial" button.

## Code Snippet:

```
code > frontend > src > components > Login.jsx > Login

1  import { useEffect, useState } from 'react'
2  import { useNavigate } from 'react-router-dom'
3
4  export default function Login({ user, onLogin }) {
5    const [email, setEmail] = useState('')
6    const [password, setPassword] = useState('')
7    const [error, setError] = useState('')
8    const [busy, setBusy] = useState(false)
9    const navigate = useNavigate()
10
11    useEffect(() => { if (user) navigate('/') }, [user, navigate])
12
13    async function handleSubmit(e) {
14      e.preventDefault()
15      setError('')
16      if (!email.trim() || !password) { setError('Email and password are required. '); return }
17      setBusy(true)
18      try { await onLogin(email.trim(), password) }
19      catch (err) { setError(err.message) }
20      finally { setBusy(false) }
21    }
22
23    return (
24      <section className="card narrow">
25        <h2>Log in</h2>
26        <form onSubmit={handleSubmit} noValidate>
27          <div className="field">
28            <label htmlFor="email">Email <span className="req">*</span></label>
29            <input id="email" type="email" value={email}
30              onChange={(e) => setEmail(e.target.value)} placeholder="you@sjsu.edu" />
31          </div>
32          <div className="field">
33            <label htmlFor="password">Password <span className="req">*</span></label>
34            <input id="password" type="password" value={password}
35              onChange={(e) => setPassword(e.target.value)} />
36          </div>
37          {error && <p className="err">{error}</p>}
38          <button type="submit" className="submit-btn" disabled={busy}>
39            {busy ? 'Logging in...' : 'Log in'}
40          </button>
41        </form>
42      </section>
43    )
44  }
```

## Output Snippet:



## Add trial

CreateRecord.jsx renders at /create. It has one input for Trial Title which is a primary field and one for Sponsor which is a secondary field plus a submit button. On submit it calls POST /api/trials. The backend gives the new row an auto-increment ID in MySQL. The user is then sent back to Home, where the new trial appears.

Code Snippet:



## Update Record:

UpdateRecord.jsx renders at /update. It reads the trial's ID from the URL e.g /update?id=2 and pre-fills the form with the current values. On submit it calls PUT /api/trials/{id}, which updates the row in MySQL. The user is sent back to Home which shows the updated values.

### Code snippet:

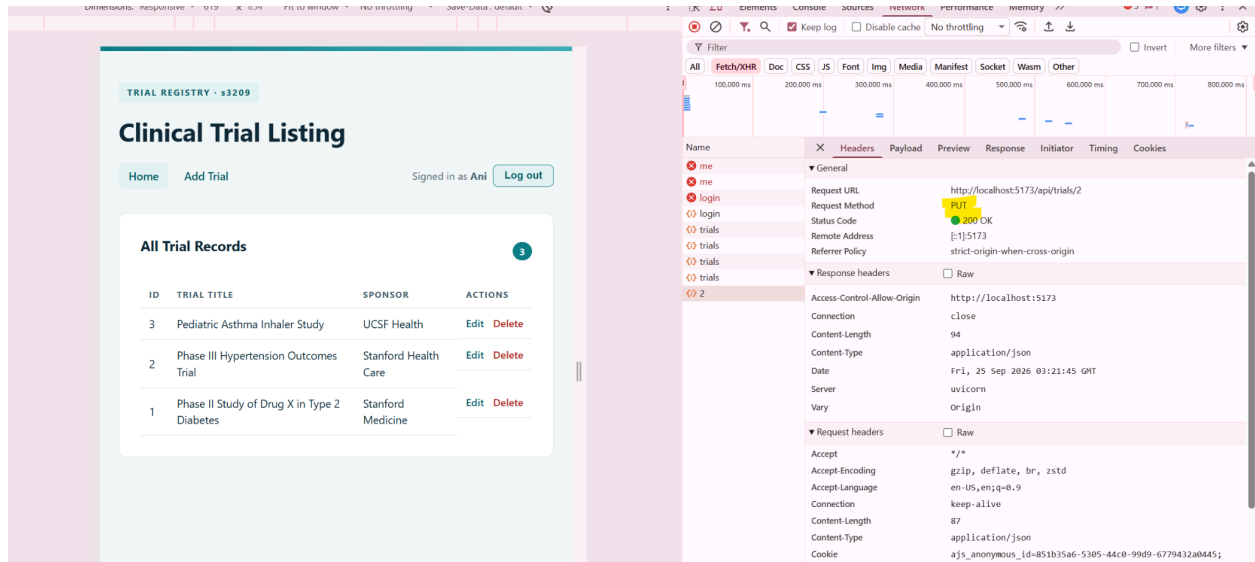
```
de > frontend > src > components > UpdateRecord.jsx > UpdateRecord
4  export default function UpdateRecord({ trials, onUpdate }) {
19
20  if (!record) {
21    return (
22      <section className="card narrow">
23        <p className="state state-error">Trial not found. Choose "Edit" from the list.</p>
24        <Link to="/">Back to list</Link>
25      </section>
26    )
27  }
28
29  async function handleSubmit(e) {
30    e.preventDefault()
31    const title = trialTitle.trim()
32    const spon = sponsor.trim()
33    if (!title || !spon) { setError('Both fields are required.');
```

load\_dotenv(Path(\_\_file\_\_).parent / '.env')

```
    return;
34    setBusy(true)
35    try {
36      await onUpdate(id, { trialTitle: title, sponsor: spon })
37      navigate('/')
38    } catch (err) {
39      setError(err.message)
40      setBusy(false)
41    }
42  }
43
44  return (
45    <section className="card narrow">
46      <h2>Update Trial #{id}</h2>
47      <form onSubmit={handleSubmit} noValidate>
48        <div className="field">
49          <label htmlFor="trialTitle">Trial Title <span className="req">*</span></label>
50          <input id="trialTitle" value={trialTitle} onChange={(e) => setTrialTitle(e.target.value)} />
51        </div>
52        <div className="field">
53          <label htmlFor="sponsor">Sponsor / Institution <span className="req">*</span></label>
54          <input id="sponsor" value={sponsor} onChange={(e) => setSponsor(e.target.value)} />
55        </div>
56        {error && <p className="err">{error}</p>}
57        <button type="submit" className="submit-btn" disabled={busy}>
58          {busy ? 'Saving...' : 'Update Trial Listing'}
59        </button>
60      </form>
61    </section>
62  )
63}
```

Do you want to import this file?

### Output Snippet:



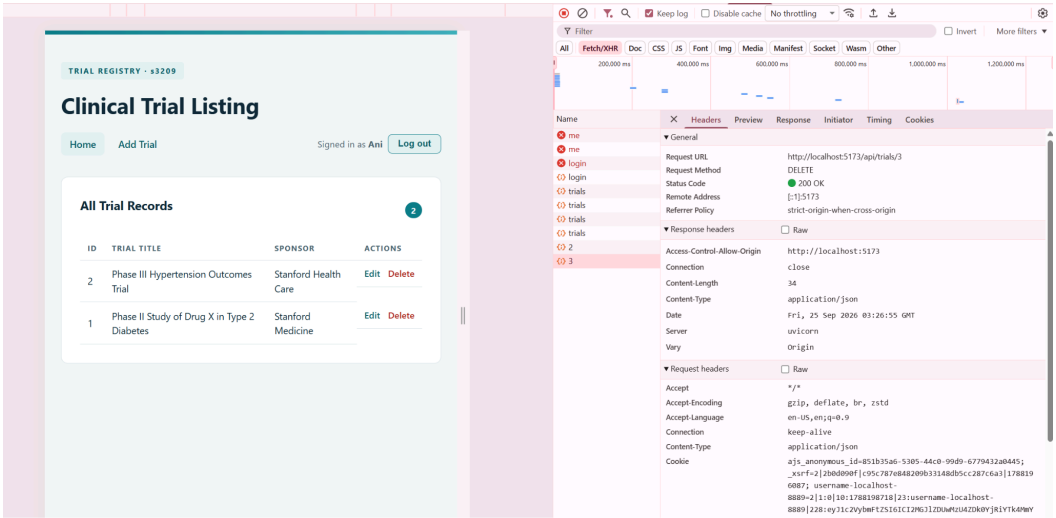
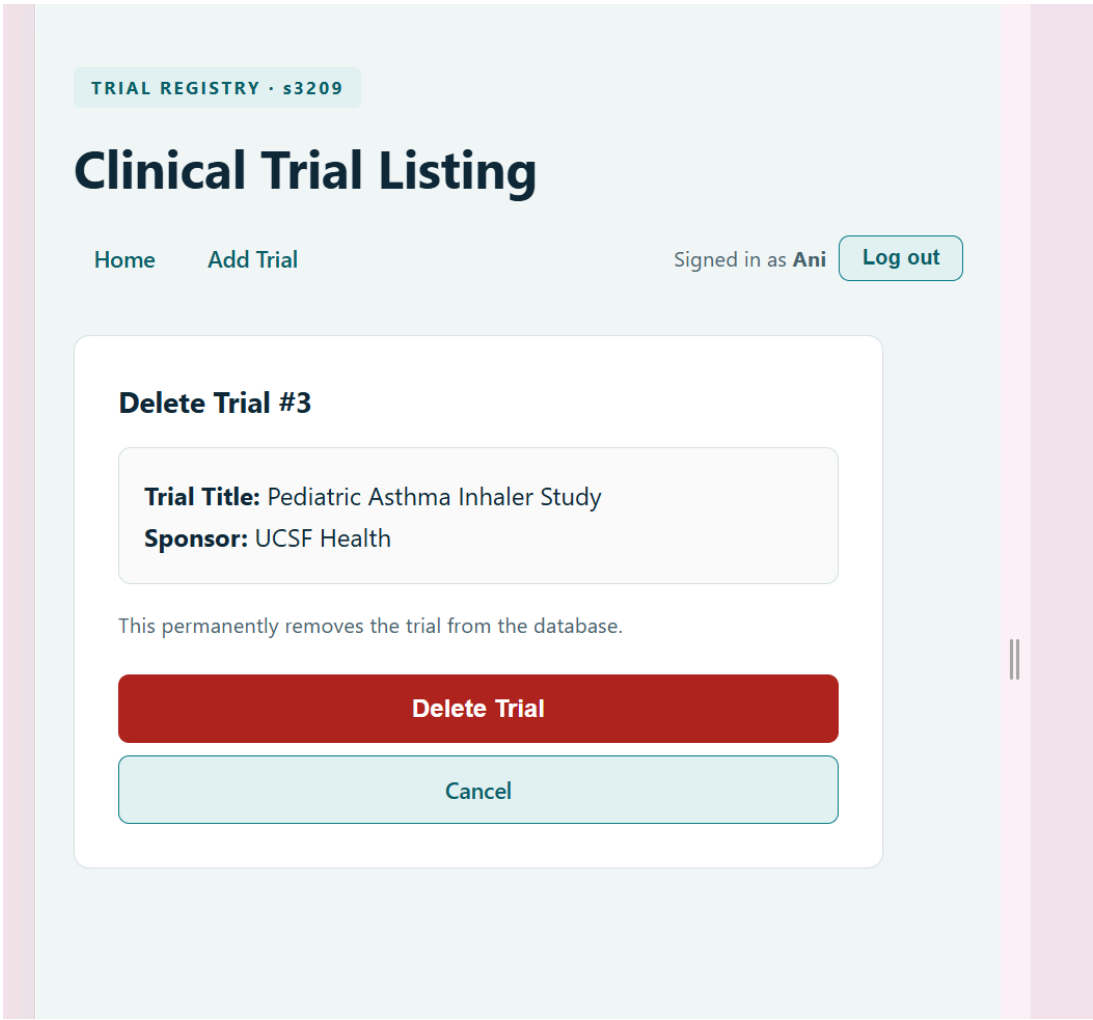
## Delete:

DeleteRecord.jsx renders at /delete. It shows the chosen trial's details and a Delete Trial button. On click it calls DELETE /api/trials/{id} which removes the row from MySQL.

## Code Snippet:

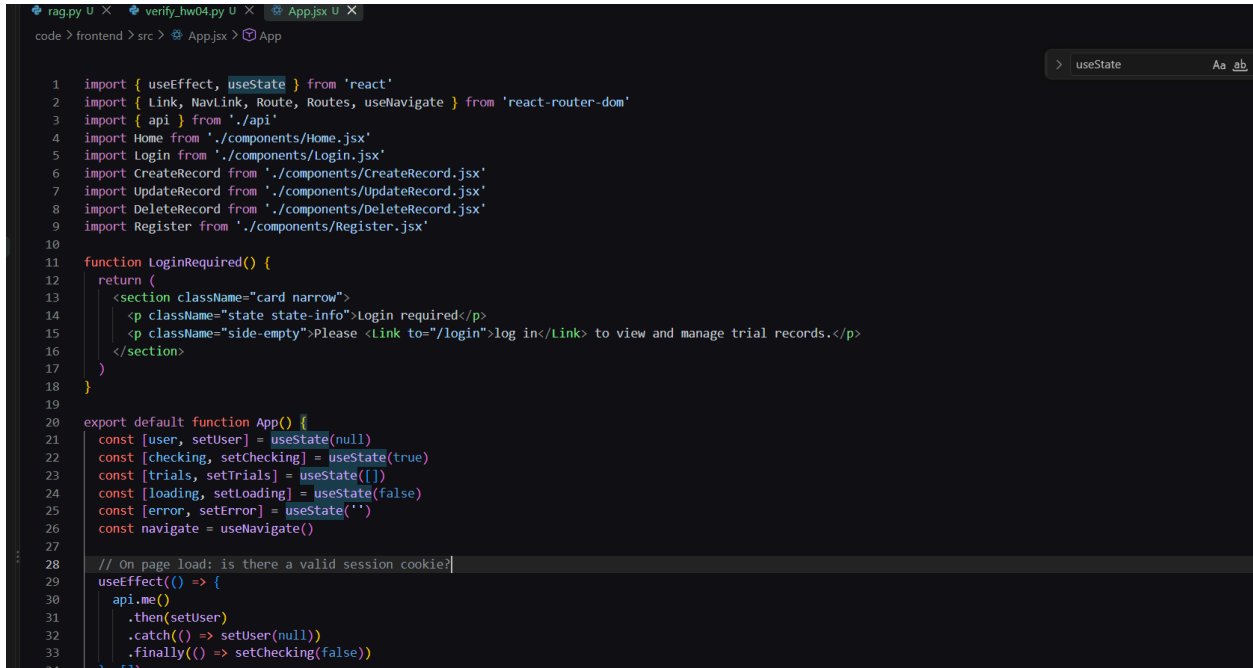
```
code / frontend / src / components / DeleteRecord.jsx / DeleteRecord
2 import { Link, useNavigate, useSearchParams } from 'react-router-dom'
3
4 export default function DeleteRecord({ trials, onDelete }) {
5   const [params] = useSearchParams()
6   const id = Number(params.get('id'))
7   const record = trials.find((t) => t.id === id)
8   const [error, setError] = useState('')
9   const [busy, setBusy] = useState(false)
10  const navigate = useNavigate()
11
12  if (!record) {
13    return (
14      <section className="card narrow">
15        <p className="state state-error">Trial not found. Choose "Delete" from the list.</p>
16        <Link to="/">Back to list</Link>
17      </section>
18    )
19  }
20
21  async function handleDelete() {
22    setBusy(true)
23    try {
24      await onDelete(id)
25      navigate('/')
26    } catch (err) {
27      setError(err.message)
28      setBusy(false)
29    }
30  }
31
32  return (
33    <section className="card narrow">
34      <h2>Delete Trial #{id}</h2>
35      <div className="record-box">
36        <p><strong>Trial Title:</strong> {record.trialTitle}</p>
37        <p><strong>Sponsor:</strong> {record.sponsor}</p>
38      </div>
39      <p className="side-empty">This permanently removes the trial from the database.</p>
40      {error && <p className="err">{error}</p>}
41      <button className="submit-btn danger-btn" onClick={handleDelete} disabled={busy}>
42        {busy ? 'Deleting...' : 'Delete Trial'}
43      </button>
44      <Link to="/" className="clear-btn cancel-link">Cancel</Link>
45    </section>
46  )
47 }
```

Output Snippet:



## Routing

App.jsx holds the trial list and the login state. It passes onAdd, onUpdate and onDelete as props to the Create, Update and Delete components. useState holds the form fields and the trial list. useEffect loads the session on page load and loads the trial list after login. react-router-dom handles the /, /login, /create, /update, and /delete routes.



```
1 import { useEffect, useState } from 'react'
2 import { Link, NavLink, Route, Routes, useNavigate } from 'react-router-dom'
3 import { api } from './api'
4 import Home from './components/Home.jsx'
5 import Login from './components/Login.jsx'
6 import CreateRecord from './components/CreateRecord.jsx'
7 import UpdateRecord from './components/UpdateRecord.jsx'
8 import DeleteRecord from './components/DeleteRecord.jsx'
9 import Register from './components/Register.jsx'
10
11 function LoginRequired() {
12   return (
13     <section className="card narrow">
14       <p className="state state-info">Login required</p>
15       <p className="side-empty">Please <Link to="/login">log in</Link> to view and manage trial records.</p>
16     </section>
17   )
18 }
19
20 export default function App() {
21   const [user, setUser] = useState(null)
22   const [checking, setChecking] = useState(true)
23   const [trials, setTrials] = useState([])
24   const [loading, setLoading] = useState(false)
25   const [error, setError] = useState('')
26   const navigate = useNavigate()
27
28   // On page load: is there a valid session cookie?
29   useEffect(() => {
30     api.me()
31       .then(setUser)
32       .catch(() => setUser(null))
33       .finally(() => setChecking(false))
34   }, [])
35 }
```

## Part 2 - MySQL Persistence and Server-Side Sessions

The database s3209\_rel holds three tables, created by SQLAlchemy in models.py and built by init\_db.py.

- trials - the domain entity. Auto-increment id, trial\_title (PF), sponsor (SF).
- users - login accounts. id, name, email (unique), password\_hash (bcrypt hash).
- sessions - server-side session store. id (the opaque session token), user\_id (foreign key to users), created\_at, expires\_at.

Database.py: Connects FastAPI to the MySQL database s3209\_rel using SQLAlchemy. The login details are read from a .env file so no password is in the code. Each request gets its own database session named db\_session\_based26 which is closed after the request.

Code snippet:

```

ode > web_application > database.py
1  import os
2  from pathlib import Path
3
4  from dotenv import load_dotenv
5  from sqlalchemy import create_engine
6  from sqlalchemy.engine import URL
7  from sqlalchemy.orm import declarative_base, sessionmaker
8
9  load_dotenv(Path(__file__).resolve().parent / ".env")
10
11  DB_URL = URL.create(
12      "mysql+pymysql",
13      username=os.getenv("DB_USER", "root"),
14      password=os.getenv("DB_PASSWORD", ""),
15      host=os.getenv("DB_HOST", "localhost"),
16      port=int(os.getenv("DB_PORT", "3306")),
17      database=os.getenv("DB_NAME", "s3209_rel"),
18  )
19
20  engine = create_engine(DB_URL, pool_pre_ping=True)
21  SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)
22  Base = declarative_base()
23
24
25  def get_db():
26      # Required name: db_session_based26
27      db_session_based26 = SessionLocal()
28      try:
29          yield db_session_based26
30      finally:
31          db_session_based26.close()

```

model.py: Defines three tables. trials holds the domain entity. users holds the login accounts, with a unique email and a hashed password. sessions holds the server-side session tokens, linked to a user with created and expiry times

```

de > web_application > models.py
1  from datetime import datetime
2
3  from sqlalchemy import Column, DateTime, ForeignKey, Integer, String
4  from sqlalchemy.orm import relationship
5
6  from database import Base
7
8
9  class Trial(Base):
10     __tablename__ = "trials"
11     id = Column(Integer, primary_key=True, autoincrement=True)
12     trial_title = Column(String(255), nullable=False) # primary field
13     sponsor = Column(String(255), nullable=False) # secondary field
14
15
16  class User(Base):
17     __tablename__ = "users"
18     id = Column(Integer, primary_key=True, autoincrement=True)
19     name = Column(String(100), nullable=False)
20     email = Column(String(255), unique=True, nullable=False)
21     password_hash = Column(String(255), nullable=False)
22     sessions = relationship("UserSession", back_populates="user",
23                             cascade="all, delete-orphan")
24
25
26  class UserSession(Base):
27     __tablename__ = "sessions"
28     id = Column(String(64), primary_key=True) # opaque session token
29     user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"),
30                     nullable=False)
31     created_at = Column(DateTime, nullable=False, default=datetime.utcnow)
32     expires_at = Column(DateTime, nullable=False)
33     user = relationship("User", back_populates="sessions")

```

## Output snippet

Running `init_db.py` created all three tables in MySQL. The output lists sessions, trials, and users, which confirms the schema was created.

```
25
26 class UserSession(Base):
    ...

Problems Output Debug Console Terminal Ports
>
Terminal
PS D:\data260-3209> cd D:\data260-3209
>> py -3.12 -m pip install -r requirements.txt
PS D:\data260-3209> Add-Content .gitignore "`ncode/web_application/.env"
PS D:\data260-3209> cd D:\data260-3209\code\web_application
>> py -3.12 init_db.py
Tables in s3209_rel: ['sessions', 'trials', 'users']
PS D:\data260-3209\code\web_application> Add-Content .gitignore "`ncode/web_application/.env"
```

## CRUD API

trials.py implements the CRUD system for the domain entity:

- POST /api/trials - add a new record
- GET /api/trials - view all records
- GET /api/trials/{id} - view a record by ID
- PUT /api/trials/{id} - update record details
- DELETE /api/trials/{id} - delete a record

Every route in trials.py requires a logged-in session (Depends(require\_user)), so no CRUD operation works without authentication.

## Server-Side Sessions

Security.py manages the user login sessions. When a user logs in the backend checks the email and password, creates a random secret token saves it in the sessions table and sends it to the browser in an HTTP-only cookie called session\_token. The cookie contains only the random token not the user's name, email or password. Whenever the user makes a request, the backend checks this token in the database to confirm that the session is valid. The session expires after 30 minutes. Passwords are protected using bcrypt so the real password is never stored. Instead, it is converted into a secure hash and saved in the password\_hash column. When the user logs in again the entered password is compared with the stored hash.

Code Snippet:

```

code > web_application > security.py
10
11 COOKIE_NAME = "session_token"
12 SESSION_MINUTES = 30
13
14
15 def hash_password(password: str) -> str:
16     return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()
17
18
19 def verify_password(password: str, password_hash: str) -> bool:
20     return bcrypt.checkpw(password.encode(), password_hash.encode())
21
22
23 def create_session(db_session_base26: Session, user: User) -> str:
24     token = secrets.token_urlsafe(32) # random, holds no user data
25     now = datetime.utcnow()
26     db_session_base26.add(UserSession(
27         id=token, user_id=user.id, created_at=now,
28         expires_at=now + timedelta(minutes=SESSION_MINUTES),
29     ))
30     db_session_base26.commit()
31     return token
32
33
34 def set_session_cookie(response, token: str):
35     response.set_cookie(
36         key=COOKIE_NAME, value=token, httponly=True, samesite="lax",
37         secure=False, # localhost is http; set True under https
38         max_age=SESSION_MINUTES * 60, path="/",
39     )
40
41
42 def get_user_from_request(request: Request, db_session_base26: Session):
43     token = request.cookies.get(COOKIE_NAME)
44     if not token:
45         return None
46     session = db_session_base26.get(UserSession, token)
47     if not session:
48         return None
49     if session.expires_at < datetime.utcnow():
50         db_session_base26.delete(session)
51         db_session_base26.commit()
52         return None
53     return session.user
54

```

## Postman API's:

### Register:

The screenshot shows the Postman interface with a POST request to `http://localhost:8509/api/auth/register`. The request body is a JSON object: `{ "name": "Postman Test", "email": "postman@sjsu.edu", "password": "pass1234" }`. The response is a 201 Created status with a JSON body: `{ "id": 2, "name": "Postman Test", "email": "postman@sjsu.edu" }`.

**Request Details:**

- Method: POST
- URL: `http://localhost:8509/api/auth/register`
- Body Type: raw
- Body Content: `{ "name": "Postman Test", "email": "postman@sjsu.edu", "password": "pass1234" }`

**Response Details:**

- Status: 201 Created
- Time: 974 ms
- Size: 189 B
- Body Type: JSON
- Body Content: `{ "id": 2, "name": "Postman Test", "email": "postman@sjsu.edu" }`

## Invalid details(Protection required)

The screenshot shows a Postman interface for a GET request to `http://localhost:8509/api/trials`. The request body is raw JSON: `{ "email": "postman@sjsu.edu", "password": "pass1234" }`. The response is a 401 Unauthorized status with a message: `{ "detail": "Login required" }`. The status bar indicates a 401 Unauthorized response, 16 ms, and 164 B.

```
GET http://localhost:8509/api/trials
```

```
{ "email": "postman@sjsu.edu", "password": "pass1234" }
```

```
{ "detail": "Login required" }
```

## Login:

The screenshot shows a Postman interface for a POST request to `http://localhost:8509/api/auth/login`. The request body is raw JSON: `{ "email": "postman@sjsu.edu", "password": "pass1234" }`. The response is a 200 OK status with a message: `{ "id": 2, "name": "Postman Test", "email": "postman@sjsu.edu" }`. The status bar indicates a 200 OK response, 774 ms, and 301 B.

```
POST http://localhost:8509/api/auth/login
```

```
{ "email": "postman@sjsu.edu", "password": "pass1234" }
```

```
{ "id": 2, "name": "Postman Test", "email": "postman@sjsu.edu" }
```

Body

Cookies 1

Headers 5

Test Results 1/1

200 OK

774 ms

301 B

Save Response

| Name          | Value                  | Domain    | Path | Expires | HttpOnly | Secure |
|---------------|------------------------|-----------|------|---------|----------|--------|
| session_token | m1WJ38AQ3nY6S_JsLYj... | localhost | /    | Session | true     | false  |

Created:

http://localhost:8509/api/trials

POST

http://localhost:8509/api/trials

Send

Docs Params Authorization Headers 9 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {"trialTitle": "Postman Created Trial", "sponsor": "Postman Labs"}
```

Body Cookies 1 Headers 4 Test Results

201 Created

43 ms

202 B

Preview Visualize

```
1 {
2   "id": 4,
3   "trialTitle": "Postman Created Trial",
4   "sponsor": "Postman Labs"
5 }
```

Get all:

http://localhost:8509/api/trials

GET

http://localhost:8509/api/trials

Docs Params Authorization Headers 7 Body Scripts Settings

Query Params

| Key | Value | Description |
|-----|-------|-------------|
| Key | Value | Description |

Body Cookies 1 Headers 4 Test Results

200 OK

Preview Visualize

```
1 [
2   {
3     "id": 4,
4     "trialTitle": "Postman Created Trial",
5     "sponsor": "Postman Labs"
6   },
7   {
8     "id": 2,
9     "trialTitle": "Phase III Hypertension Outcomes Trial",
10    "sponsor": "Stanford Health Care"
11  },
12  {
13    "id": 1,
14    "trialTitle": "Phase II Study of Drug X in Type 2 Diabetes",
15    "sponsor": "Stanford Medicine"
16  }
17 ]
```

## Get Particular Id:

Postman interface showing a GET request to `http://localhost:8509/api/trials/4`. The response is a JSON object with the following structure:

```
{  "id": 4,  "trialTitle": "Postman Created Trial",  "sponsor": "Postman Labs"}
```

## Delete:

Postman interface showing a DELETE request to `http://localhost:8509/api/trials/4`. The response is a JSON object with the following structure:

```
{  "message": "Trial deleted",  "id": 4}
```

Postman interface showing a GET request to `http://localhost:8509/api/trials`. The response is a JSON array of two trial objects:

```
[  {    "id": 2,    "trialTitle": "Phase III Hypertension Outcomes Trial",    "sponsor": "Stanford Health Care"  },  {    "id": 1,    "trialTitle": "Phase II Study of Drug X in Type 2 Diabetes",    "sponsor": "Stanford Medicine"  }]
```

Postman interface showing a GET request to `http://localhost:8509/api/trials`. The response is a JSON array of two trial objects:

```
[  {    "id": 2,    "trialTitle": "Phase III Hypertension Outcomes Trial",    "sponsor": "Stanford Health Care"  },  {    "id": 1,    "trialTitle": "Phase II Study of Drug X in Type 2 Diabetes",    "sponsor": "Stanford Medicine"  }]
```

Update:

The image shows a Postman interface for a PUT request. The URL is `http://localhost:8509/api/trials/5004`. The method is `PUT`. The body is set to `raw` with a JSON payload: `{"trialTitle": "Postman Updated Trial", "sponsor": "Postman Labs Updated"}`. The response is displayed in the bottom panel, showing a JSON object with `id: 5004`, `trialTitle: "Postman Updated Trial"`, and `sponsor: "Postman Labs Updated"`.

```
1 {"trialTitle": "Postman Updated Trial", "sponsor": "Postman Labs Updated"}
```

```
1 {
2   "id": 5004,
3   "trialTitle": "Postman Updated Trial",
4   "sponsor": "Postman Labs Updated"
5 }
```

Testing:

| Operation            | Method | Status |
|----------------------|--------|--------|
| Register             | POST   | 201    |
| Blocked before login | GET    | 401    |
| Login                | POST   | 200    |
| Create               | POST   | 201    |
| View All             | GET    | 200    |
| Update               | PUT    | 200    |
| Delete               | DELETE | 200    |

Sql:

```
mysql> USE s3209_rel;
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_s3209_rel |
+-----+
| sessions             |
| trials               |
| users                |
+-----+
3 rows in set (0.02 sec)

mysql> SELECT * FROM trials;
+-----+-----+-----+
| id | trial_title                                     | sponsor           |
+-----+-----+-----+
| 1  | Phase II Study of Drug X in Type 2 Diabetes | Stanford Medicine |
| 2  | Phase III Hypertension Outcomes Trial       | Stanford Health Care |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM users;
+-----+-----+-----+-----+
| id | name      | email           | password_hash |
+-----+-----+-----+-----+
| 1  | Ani       | ani@sjsu.edu   | $2b$12$f9s9G1aRDjDGWNumThePceW0tMKtAqcPWuN657RYFTbKRL9Mtb880 |
| 2  | Postman Test | postman@sjsu.edu | $2b$12$cSwBwFRfYgg1700hydXxj.kxhbnjvZD5v.cL/.EjkUNbom2auVSYW |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM sessions;
+-----+-----+-----+-----+
| id | user_id | created_at      | expires_at |
+-----+-----+-----+-----+
| m1WJ38AQ3nY6S_lsLYjLGMuAjBWqJSnHqyVhRc80H8U | 2 | 2026-09-25 03:59:46 | 2026-09-25 04:29:46 |
| Q1xTTNI0U9jqux8jE1GzHwYR0c0r_wN4RkjeY7tEG8g | 1 | 2026-09-25 03:14:25 | 2026-09-25 03:44:25 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

**Directory :**

```
PS D:\data260-3209\code> dir web_application
```

```
Directory: D:\data260-3209\code\web_application
```

| Mode    | LastWriteTime      | Length | Name        |
|---------|--------------------|--------|-------------|
| d-----  | 9/17/2026 10:57 AM |        | templates   |
| d-----  | 9/24/2026 7:48 PM  |        | __pycache__ |
| -a----- | 9/24/2026 7:37 PM  | 28     | .gitignore  |
| -a----- | 9/17/2026 10:18 AM | 7563   | app.js      |
| -a----- | 9/24/2026 7:46 PM  | 4784   | auth.py     |
| -a----- | 9/24/2026 7:48 PM  | 895    | database.py |
| -a----- | 9/17/2026 10:18 AM | 5602   | index.html  |
| -a----- | 9/24/2026 7:37 PM  | 227    | init_db.py  |
| -a----- | 9/24/2026 7:47 PM  | 553    | main.py     |
| -a----- | 9/24/2026 7:36 PM  | 1277   | models.py   |
| -a----- | 9/17/2026 10:18 AM | 160    | nginx.conf  |
| -a----- | 9/24/2026 7:46 PM  | 2238   | security.py |
| -a----- | 9/17/2026 10:18 AM | 5542   | styles.css  |
| -a----- | 9/24/2026 7:47 PM  | 2809   | trials.py   |

```
PS D:\data260-3209\code> dir frontend\src
```

```
Directory: D:\data260-3209\code\frontend\src
```

| Mode    | LastWriteTime     | Length | Name       |
|---------|-------------------|--------|------------|
| d-----  | 9/24/2026 9:53 PM |        | components |
| -a----- | 9/24/2026 9:55 PM | 1231   | api.js     |
| -a----- | 9/24/2026 8:01 PM | 1986   | app.css    |
| -a----- | 9/24/2026 9:58 PM | 4428   | App.jsx    |
| -a----- | 9/24/2026 7:57 PM | 384    | main.jsx   |

```
PS D:\data260-3209\code> dir frontend\src\components
```

```
Directory: D:\data260-3209\code\frontend\src\components
```

| Mode    | LastWriteTime      | Length | Name             |
|---------|--------------------|--------|------------------|
| -a----- | 9/24/2026 7:59 PM  | 1775   | CreateRecord.jsx |
| -a----- | 9/24/2026 8:00 PM  | 1502   | DeleteRecord.jsx |
| -a----- | 9/24/2026 7:59 PM  | 1674   | Home.jsx         |
| -a----- | 9/24/2026 7:59 PM  | 1639   | Login.jsx        |
| -a----- | 9/24/2026 10:04 PM | 2244   | Register.jsx     |
| -a----- | 9/24/2026 8:00 PM  | 2193   | UpdateRecord.jsx |

```
PS D:\data260-3209\code>
```

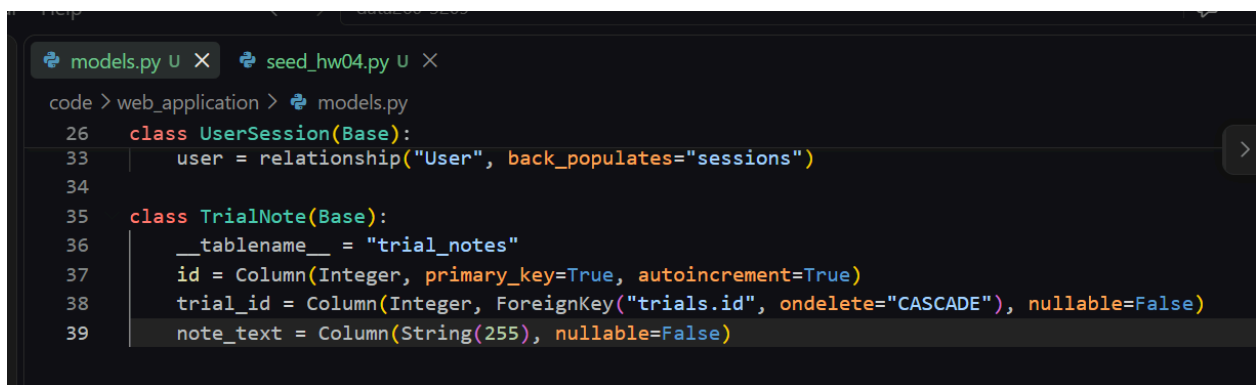
## Part 3 - N+1 Measurement and Query Tuning

***This section seeds a large dataset, demonstrates the classic N+1 query problem on a naive endpoint fixes it with eager loading, measures the before/after performance across three page sizes, adds a database index, and confirms both endpoints with Postman.***

## 1) Seed the database

A related table, `trial_notes`, was added to hold test data for the N+1 experiment. Each note links to one trial through `trial_id`.

Code Snippet:



```
code > web_application > models.py
26 class UserSession(Base):
33     user = relationship("User", back_populates="sessions")
34
35 class TrialNote(Base):
36     __tablename__ = "trial_notes"
37     id = Column(Integer, primary_key=True, autoincrement=True)
38     trial_id = Column(Integer, ForeignKey("trials.id", ondelete="CASCADE"), nullable=False)
39     note_text = Column(String(255), nullable=False)
```

A seed script (`seed_hw04.py`) generates 5,000 trial rows and 200 related `trial_notes` rows using `SEED = 3209`, as required.

```

models.py U X seed_hw04.py U X
code > web_application > seed_hw04.py
from models import Trial, TrialNote
5
6 SEED = 3209
7 random.seed(SEED)
8
9 SPONSORS = ["Stanford Medicine", "Mayo Clinic", "UCSF Health", "Johns Hopkins",
10            "Cleveland Clinic", "Mass General", "Duke Health", "NYU Langone"]
11 STATUSES = ["Phase I", "Phase II", "Phase III", "Observational"]
12
13 Base.metadata.create_all(bind=engine)
14
15 db_session_based26 = SessionLocal()
16
17 try:
18     db_session_based26.query(TrialNote).delete()
19     db_session_based26.query(Trial).delete()
20     db_session_based26.commit()
21
22     trials = []
23     for i in range(1, 5001):
24         trials.append(Trial(
25             trial_title=f"Study #{i}",
26             sponsor=random.choice(SPONSORS),
27         ))
28     db_session_based26.add_all(trials)
29     db_session_based26.commit()
30
31     trial_ids = [t.id for t in trials]
32
33     notes = []
34     for i in range(200):
35         notes.append(TrialNote(
36             trial_id=random.choice(trial_ids),
37             note_text=f"Note {i + 1}: monitoring update for this trial.",
38         ))
39     db_session_based26.add_all(notes)
40     db_session_based26.commit()
41
42     print(f"Seeded {len(trials)} trials and {len(notes)} trial_notes (SEED={SEED}).")
43 finally:
44     db_session_based26.close()

```

Output snippet:

```

Problems Output Debug Console Terminal Ports
● PS D:\data260-3209\code> Test-Path frontend\src\components\Register.jsx
True
PS D:\data260-3209\code> cd D:\data260-3209\code\web_application
● >> py -3.12 seed_hw04.py
Seeded 5000 trials and 200 trial_notes (SEED=3209).
○ PS D:\data260-3209\code\web_application>

```

Row counts in MySQL

```

PS D:\data260-3209\code> mysql -u root -p -e "SELECT COUNT(*) FROM s3209_rel.trials; SELECT COUNT(*) FROM s3209_rel.trial_notes;"
Enter password: *****
+-----+
| COUNT(*) |
+-----+
|      5000 |
+-----+
+-----+
| COUNT(*) |
+-----+
|       200 |
+-----+
PS D:\data260-3209\code>

```

2) Related data included in the list response

Each trial\_notes row is linked to a trial via trial\_id and both the naive and fixed list endpoints below include each trial's notes in their JSON response satisfying the requirement that related data be returned with the primary entity

### 3) Build the naive N+1 endpoint

[trial.py](#)

Intentionally naive endpoint. It runs 1 query for the trial list, then 1 extra query per trial to fetch its notes the classic N+1 pattern.

Code Snippet :

```
@router.get("/naive/list")
def list_trials_naive(page_size: int = 10, db_session_based26: Session = Depends(get_db)):
    """Intentionally naive: 1 query for trials + 1 extra query PER trial for its notes."""
    trials = (
        db_session_based26.query(Trial)
        .order_by(Trial.id)
        .limit(page_size)
        .all()
    ) # query #1

    result = []
    for t in trials:
        notes = (
            db_session_based26.query(TrialNote)
            .filter(TrialNote.trial_id == t.id)
            .all()
        ) # 1 extra query per trial - this is the N+1 problem
        result.append({
            "id": t.id,
            "trialTitle": t.trial_title,
            "sponsor": t.sponsor,
            "notes": [n.note_text for n in notes],
        })
    return result
```

A SQLAlchemy event listener that counts every SQL statement sent to MySQL used to measure the N+1 effect.

Query counter:

```
34
35
36 query_count = {"count": 0}
37
38 @event.listens_for(engine, "before_cursor_execute")
39 def _count_queries(conn, cursor, statement, parameters, context, executemany):
40     query_count["count"] += 1
```

Output:

| Page Size | Queries |
|-----------|---------|
| 10        | 13      |
| 50        | 53      |
| 200       | 203     |

Query count scales almost 1-to-1 with page size confirming the N+1 problem: each additional trial adds one more query.

Output- manual verification

```
PS D:\data260-3209\code\web_application> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/query-count" -Method GET -WebSession $session

queries
-----
      13

PS D:\data260-3209\code\web_application> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/reset-query-count" -Method
```

```
-WebSession $session
>> Invoke-RestMethod -Uri "http://localhost:8509/api/trials/naive/list?page_size=200" -Method GET -WebSession $session
>> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/query-count" -Method GET -WebSession $session

203

PS D:\data260-3209\code\web_application>
```

```
PS D:\data260-3209\code\web_application> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/reset-query-count" -Method GET -WebSession $session
>> Invoke-RestMethod -Uri "http://localhost:8509/api/trials/naive/list?page_size=50" -Method GET -WebSession $session
>> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/query-count" -Method GET -WebSession $session

53

PS D:\data260-3209\code\web_application> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/reset-query-count" -Method GET -WebSession $session
>> Invoke-RestMethod -Uri "http://localhost:8509/api/trials/naive/list?page_size=200" -Method GET -WebSession $session
>> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/query-count" -Method GET -WebSession $session
```

#### 4) Measure the naive endpoint

The naive list endpoint (GET /api/trials/naive/list) was tested at page sizes 10, 50, and 200, with 30 requests recorded at each size. A query counter (database.py, using a SQLAlchemy before\_cursor\_execute event) recorded the exact number of SQL statements per request, and time.perf\_counter() measured latency per request in measure\_hw04.py.

```
models.py U X measure_hw04.py U X seed_hw04.py U X trials.py U X database.py U X main.py M X
code > web_application > measure_hw04.py
7
8 BASE = "http://localhost:8509"
9 EMAIL = "ani@sjsu.edu"
10 PASSWORD = "pass1234"
11 PAGE_SIZES = [10, 50, 200]
12 RUNS_PER_CONFIG = 30
13
14 OUT_DIR = Path(__file__).resolve().parents[2] / "reports" / "hw04" / "raw"
15 OUT_DIR.mkdir(parents=True, exist_ok=True)
16
17
18 def login():
19     s = requests.Session()
20     r = s.post(f"{BASE}/api/auth/login", json={"email": EMAIL, "password": PASSWORD})
21     r.raise_for_status()
22     return s
23
24
25 def reset_counter(session):
26     session.post(f"{BASE}/api/debug/reset-query-count").raise_for_status()
27
28
29 def get_counter(session):
30     return session.get(f"{BASE}/api/debug/query-count").json()["queries"]
31
32
33 def measure(session, version, page_size):
34     rows = []
35     endpoint = f"{BASE}/api/trials/{version}/list?page_size={page_size}"
36
37     for i in range(RUNS_PER_CONFIG):
38         reset_counter(session)
39         start = time.perf_counter()
40         r = session.get(endpoint)
41         elapsed_ms = (time.perf_counter() - start) * 1000
42         r.raise_for_status()
43         queries = get_counter(session)
44
45         rows.append({
46             "run": i + 1,
47             "page_size": page_size,
48             "version": version,
49             "sql_queries": queries,
50             "latency_ms": round(elapsed_ms, 2),
51         })
52         print(f"version:{version} | page_size:{page_size:3d} | run {i+1:2d}/30 "
53               f"| queries={queries:3d} | {elapsed_ms:7.2f} ms")
54
55     return rows
56
```

```

code > web_application > measure_hw04.py
67 def summarize(rows):
68     return {
69         "sql_queries": rows[0]["sql_queries"],
70         "p50": round(percentile(latencies, 50), 2),
71         "p95": round(percentile(latencies, 95), 2),
72         "p99": round(percentile(latencies, 99), 2),
73     }
74
75
76
77 def main():
78     session = login()
79     all_rows = []
80     summary_rows = []
81
82     for version in ["naive", "fixed"]:
83         for page_size in PAGE_SIZES:
84             rows = measure(session, version, page_size)
85             all_rows.extend(rows)
86             summary = summarize(rows)
87             summary_rows.append({
88                 "page_size": page_size,
89                 "version": version,
90                 "sql_stmts_per_req": summary["sql_queries"],
91                 "p50_ms": summary["p50"],
92                 "p95_ms": summary["p95"],
93                 "p99_ms": summary["p99"],
94             })
95
96     raw_path = OUT_DIR / "n_plus_1_raw.csv"
97     with open(raw_path, "w", newline="") as f:
98         writer = csv.DictWriter(f, fieldnames=all_rows[0].keys())
99         writer.writeheader()
100         writer.writerows(all_rows)
101
102     summary_path = OUT_DIR / "n_plus_1_summary.csv"
103     with open(summary_path, "w", newline="") as f:
104         writer = csv.DictWriter(f, fieldnames=summary_rows[0].keys())
105         writer.writeheader()
106         writer.writerows(summary_rows)
107
108     print(f"\nSaved {len(all_rows)} raw rows to {raw_path}")
109     print(f"Saved summary table to {summary_path}")
110     for row in summary_rows:
111         print(row)
112
113
114 if __name__ == "__main__":
115     main()

```

| Page Size | SQL stmts/req | p50(ms) | p95(ms) | p99(ms) |
|-----------|---------------|---------|---------|---------|
| 10        | 13            | 22.3    | 31.21   | 37.71   |
| 50        | 53            | 62.69   | 67.8    | 70.82   |
| 200       | 203           | 524.23  | 1060.91 | 1293.8  |

The query count grows almost 1-to-1 with page size (13 → 53 → 203), confirming the N+1 pattern: one query for the trial list, plus one extra query per trial for its notes, plus a small constant overhead from the session-authentication lookup. Latency grows even faster than the query count, because at page\_size=200 the database and connection pool come under real load, causing some requests to take over a second (p99 = 1293.84 ms).

## 5) Fix the N+1 issue

The fixed endpoint (GET /api/trials/fixed/list) uses SQLAlchemy's `selectinload(Trial.notes)` to eager-load each trial's notes in a second, single `IN(...)` query, instead of one query per trial.

```
@router.get("/fixed/list")
def list_trials_fixed(page_size: int = 10, db_session_based26: Session = Depends(get_db)):
    """Fixed: eager-loads notes with the trials in a small, constant number of queries."""
    trials = (
        db_session_based26.query(Trial)
        .options(selectinload(Trial.notes))
        .order_by(Trial.id)
        .limit(page_size)
        .all()
    ) # query #1: trials, query #2: all their notes in one IN(...) query

    return [
        {
            "id": t.id,
            "trialTitle": t.trial_title,
            "sponsor": t.sponsor,
            "notes": [n.note_text for n in t.notes],
        }
        for t in trials
    ]
```

```
models.py U X  seed_hw04.py U X  trials.py U X  database.py U X  main.py M X
code > web_application > models.py

1  from datetime import datetime
2
3  from sqlalchemy import Column, DateTime, ForeignKey, Integer, String
4  from sqlalchemy.orm import relationship
5
6  from database import Base
7
8
9  class Trial(Base):
10     __tablename__ = "trials"
11     id = Column(Integer, primary_key=True, autoincrement=True)
12     trial_title = Column(String(255), nullable=False) # primary field
13     sponsor = Column(String(255), nullable=False) # secondary field
14     notes = relationship("TrialNote", cascade="all, delete-orphan")
15
16
```

```

● PS D:\data260-3209\code\web_application> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/query-count" -Method GET -WebSession $session

queries
-----
4

PS D:\data260-3209\code\web_application>
○ PS D:\data260-3209\code\web_application>
○ PS D:\data260-3209\code\web_application>
○ PS D:\data260-3209\code\web_application> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/reset-query-count" -Method POST -WebSession $session
>> Invoke-RestMethod -Uri "http://localhost:8509/api/trials/fixedList?page_size=10" -Method GET -WebSession $session
● >> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/query-count" -Method GET -WebSession $session

queries
-----
0

4

PS D:\data260-3209\code\web_application> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/reset-query-count" -Method POST -WebSession $session
>> Invoke-RestMethod -Uri "http://localhost:8509/api/trials/fixedList?page_size=50" -Method GET -WebSession $session
● >> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/query-count" -Method GET -WebSession $session

```

```
PS D:\data260-3209\code\web_application> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/reset-query-count" -Method POST -WebSession $session
>> Invoke-RestMethod -Uri "http://localhost:8509/api/trials/fixes/list?page_size=50" -Method GET -WebSession $session
>> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/query-count" -Method GET -WebSession $session

4

PS D:\data260-3209\code\web_application>
O PS D:\data260-3209\code\web_application> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/reset-query-count" -Method POST -WebSession $session
Invoke-RestMethod : The URI "http://localhost:8509/api/debug/reset-query-count" is not a valid URI.
Invoke-RestMethod : The URI "http://localhost:8509/api/debug/reset-query-count" is not a valid URI.
```

```
PS D:\data260-3209\code\web_application> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/reset-query-count" -Method POST -WebSession $session
>> Invoke-RestMethod -Uri "http://localhost:8509/api/trials/fixed/list?page_size=200" -Method GET -WebSession $session
>> Invoke-RestMethod -Uri "http://localhost:8509/api/debug/query-count" -Method GET -WebSession $session

4

PS D:\data260-3209\code\web_application>
```

The same 30-request measurement was repeated at page sizes 10, 50, and 200:

| Page Size | SQL stmts/req | p50 (ms) | p95 (ms) | p99 (ms) |
|-----------|---------------|----------|----------|----------|
| 10        | 4             | 35.07    | 62.96    | 71.54    |
| 50        | 4             | 32.53    | 53.07    | 78.0     |
| 200       | 4             | 43.26    | 51.34    | 97.81    |

The query count stays flat at 4 regardless of page size (1 session lookup + 1 trial query + 1 notes query + 1 constant overhead query), proving the fix removed the per-row query cost entirely.

Quick side-by-side comparison of query counts only:

| Page Size | Naive Queries | Fixed Queries |
|-----------|---------------|---------------|
| 10        | 13            | 4             |
| 50        | 53            | 4             |
| 200       | 203           | 4             |

## 6) Raw data saved

All 180 individual request measurements (3 page sizes × 2 versions × 30 requests) were saved to reports/hw04/raw/n\_plus\_1\_raw.csv, and the summary table above was saved to reports/hw04/raw/n\_plus\_1\_summary.csv, generated by measure\_hw04.py.

```
Saved 180 raw rows to D:\data260-3209\reports\hw04\raw\n_plus_1_raw.csv
Saved summary table to D:\data260-3209\reports\hw04\raw\n_plus_1_summary.csv
{'page_size': 10, 'version': 'naive', 'sql_stmts_per_req': 13, 'p50_ms': 22.3, 'p95_ms': 31.21, 'p99_ms': 37.71}
{'page_size': 50, 'version': 'naive', 'sql_stmts_per_req': 53, 'p50_ms': 62.69, 'p95_ms': 67.8, 'p99_ms': 70.82}
{'page_size': 200, 'version': 'naive', 'sql_stmts_per_req': 203, 'p50_ms': 524.23, 'p95_ms': 1060.91, 'p99_ms': 1293.84}
{'page_size': 10, 'version': 'fixed', 'sql_stmts_per_req': 4, 'p50_ms': 35.07, 'p95_ms': 62.96, 'p99_ms': 71.54}
{'page_size': 50, 'version': 'fixed', 'sql_stmts_per_req': 4, 'p50_ms': 32.53, 'p95_ms': 53.07, 'p99_ms': 78.0}
{'page_size': 200, 'version': 'fixed', 'sql_stmts_per_req': 4, 'p50_ms': 43.26, 'p95_ms': 51.34, 'p99_ms': 97.81}
PS D:\data260-3209\code\web_application>
```

```
{'page_size': 200, 'version': 'fixed', 'sGet-Content D:\data260-3209\reports\h
>> (Import-Csv D:\data260-3209\reports\hw04\raw\n_plus_1_raw.csv).Count
run,page_size,version,sql_queries,latency_ms
1,10,naive,13,30.61
2,10,naive,13,23.4
3,10,naive,13,21.07
4,10,naive,13,25.29
180
PS D:\data260-3209\code\web_application>
```

## 7) Speed-up analysis

| Page Size | Naive p50(ms) | Fixed p50(ms) | Speed-up)                           |
|-----------|---------------|---------------|-------------------------------------|
| 10        | 22.3          | 35.07         | fixed is ~0.6x<br>(slightly slower) |
| 50        | 62.69         | 32.53         | fixed is ~1.9x faster               |
| 200       | 524.23        | 43.26         | fixed is ~12.1x<br>faster           |

At page\_size=10, the fixed version is not faster. With only a few extra queries (13 vs. 4), the naive version's overhead is small enough that the fixed version's own setup cost (building the `IN(...)` batch query) is not worth it yet.

As page size grows, the speed-up increases sharply. This is because the naive version's cost grows linearly with the number of rows — each additional trial adds one more full round trip to the database. The fixed version's cost stays constant at 4 queries no matter how many trials are returned, since it batches all related notes into a single query. So the gap between the two widens as N grows, which is why the speed-up climbs from 0.6x at N=10 to 12.1x at N=200

## 8) Adding An Index

### Adding an index

Before adding an index, SHOW INDEX FROM trials showed only the PRIMARY key on id. Searching by sponsor required MySQL to scan every row.

EXPLAIN SELECT \* FROM trials WHERE trial\_title LIKE '%Diabetes%' OR sponsor LIKE '%Diabetes%' showed type: ALL, key: NULL, and rows: 4980, meaning MySQL scanned all 4,980 rows to find matches, a full table scan.

After running CREATE INDEX idx\_trials\_sponsor ON trials(sponsor), an exact-match query was tested instead of the LIKE '%...%' search, since a leading-wildcard LIKE cannot use a normal B-tree index. EXPLAIN SELECT \* FROM trials WHERE sponsor = 'Stanford Medicine' then showed type: ref, key: idx\_trials\_sponsor, and rows: 631 — MySQL used the new index to jump directly to the ~631 matching rows instead of scanning all 4,980.

```
PS D:\data260-3209\code> mysql -u root -p -e "SHOW INDEX FROM s3209_rel.trial_notes; SHOW INDEX FROM s3209_rel.trials;"
Enter password: *****
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| trial_notes | 0 | PRIMARY | 1 | id | A | 200 | NULL | NULL | NULL | BTREE | | | YES | NULL |
| trial_notes | 1 | trial_id | 1 | trial_id | A | 194 | NULL | NULL | NULL | BTREE | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| trials | 0 | PRIMARY | 1 | id | A | 4980 | NULL | NULL | NULL | BTREE | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
PS D:\data260-3209\code> mysql -u root -p s3209_rel -e "EXPLAIN SELECT * FROM trials WHERE trial_title LIKE '%Diabetes%' OR sponsor LIKE '%Diabetes%';"
Enter password: *****
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | trials | NULL | ALL | NULL | NULL | NULL | NULL | 4980 | 20.99 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
PS D:\data260-3209\code> mysql -u root -p s3209_rel -e "CREATE INDEX idx_trials_sponsor ON trials(sponsor);"
Enter password: *****
PS D:\data260-3209\code> mysql -u root -p s3209_rel -e "EXPLAIN SELECT * FROM trials WHERE sponsor = 'Stanford Medicine';"
Enter password: *****
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | trials | NULL | ref | idx_trials_sponsor | idx_trials_sponsor | 1022 | const | 631 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
PS D:\data260-3209\code> mysql -u root -p -e "SHOW INDEX FROM s3209_rel.trials;"
Enter password: *****
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| trials | 0 | PRIMARY | 1 | id | A | 4980 | NULL | NULL | NULL | BTREE | | | YES | NULL |
| trials | 1 | idx_trials_sponsor | 1 | sponsor | A | 8 | NULL | NULL | NULL | BTREE | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
PS D:\data260-3209\code>
```

What changed: the query plan switched from a full table scan (type: ALL, all rows read) to an index lookup (type: ref, only matching rows read). This is why indexes matter most on columns used in WHERE filters — without one, every search costs a full scan regardless of table size; with one, the cost only scales with how many rows actually match.

## 9)Postman

Fixed endpoint- 3 requests

Q Search

InviteUpgrade

No Environment

GET GET dataGET GET dataOverviewPOST Post data:POST http://localGET http://localhDEL http://localhPOST Naive logGET Naive endpc+No Environment

My Collection > Naive endpoint — 3 requestsSaveShare

GEThttp://localhost:8509/api/trials/fixe/list?page\_size=10Send

DocsParamsAuthorizationHeaders 7BodyScriptsSettingsCo

Query Params

| Key                                           | Value | Description | Bulk Edit |
|-----------------------------------------------|-------|-------------|-----------|
| <input checked="" type="checkbox"/> page_size | 10    |             |           |
| Key                                           | Value | Description |           |

BodyCookies 1Headers 4Test Results🔄

200 OK • 34 ms • 944 B🌐📄 Save Respons

{ } JSONPreviewVisualize

```
1  [
2    {
3      "id": 5,
4      "trialTitle": "Phase II Study #1",
5      "sponsor": "Mass General",
6      "notes": []
7    },
8    {
9      "id": 6,
10     "trialTitle": "Phase II Study #2",
11     "sponsor": "Duke Health",
12     "notes": []
13   },
14   {
15     "id": 7,
16     "trialTitle": "Phase III Study #3",
17     "sponsor": "NYU Langone",
18     "notes": []
19   },
20   {
21     "id": 8,
22     "trialTitle": "Observational Study #4",
23     "sponsor": "UCSF Health",
24     "notes": []
25   },
26   {
27     "id": 9,
28     "trialTitle": "Observational Study #5",
29     "sponsor": "Cleveland Clinic",
30     "notes": []
31   },
32   {
33     "id": 10.
```

My Collection > Naive endpoint — 3 requests

Save Share

GET http://localhost:8509/api/trials/fixe/list?page\_size=50 Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

| Query Params                                  |       |             |           |
|-----------------------------------------------|-------|-------------|-----------|
| Key                                           | Value | Description | Bulk Edit |
| <input checked="" type="checkbox"/> page_size | 50    |             |           |
| Key                                           | Value | Description |           |

Body Cookies 1 Headers 4 Test Results 200 OK 36 ms 4.13 KB Save Response

JSON Preview Visualize

```
1  [
2      {
3          "id": 5,
4          "trialTitle": "Phase II Study #1",
5          "sponsor": "Mass General",
6          "notes": []
7      },
8      {
9          "id": 6,
10         "trialTitle": "Phase II Study #2",
11         "sponsor": "Duke Health",
12         "notes": []
13     },
14     {
15         "id": 7,
16         "trialTitle": "Phase III Study #3",
17         "sponsor": "NYU Langone",
18         "notes": []
19     },
20     {
21         "id": 8,
22         "trialTitle": "Observational Study #4",
23         "sponsor": "UCSF Health",
24         "notes": []
25     },
26     {
27         "id": 9,
28         "trialTitle": "Observational Study #5",
29         "sponsor": "Cleveland Clinic",
30         "notes": []
31     },
32     {
33         "id": 10,
34         "trialTitle": "Phase III Study #6",
```

My Collection > Naive endpoint — 3 requests

GET

http://localhost:8509/api/trials/fixed/list?page\_size=200

Send

Docs

Params

Authorization

Headers 7

Body

Scripts

Settings

Cookies

Query Params

| Key         | Value | Description | Bulk Edit | ... |
|-------------|-------|-------------|-----------|-----|
| ✓ page_size | 200   |             |           |     |
| Key         | Value | Description |           |     |

Body

Cookies 1

Headers 4

Test Results

200 OK • 69 ms • 16.43 KB

Save Response

{ } JSON

Preview

Visualize

```
262     "trialTitle": "Phase II Study #44",
263     "sponsor": "NYU Langone",
264     "notes": []
265   },
266   {
267     "id": 49,
268     "trialTitle": "Phase III Study #45",
269     "sponsor": "UCSF Health",
270     "notes": []
271   },
272   {
273     "id": 50,
274     "trialTitle": "Observational Study #46",
275     "sponsor": "Mayo Clinic",
276     "notes": []
277   },
278   {
279     "id": 51,
280     "trialTitle": "Phase I Study #47",
281     "sponsor": "Duke Health",
282     "notes": []
283   },
284   {
285     "id": 52,
286     "trialTitle": "Observational Study #48",
287     "sponsor": "Johns Hopkins",
288     "notes": []
289   },
290   {
291     "id": 53,
292     "trialTitle": "Phase II Study #49",
293     "sponsor": "Mass General"
```

Naive endpoint: 3 requests

My Collection > Naive endpoint — 3 requests Save Share

GET http://localhost:8509/api/trials/naive/list?page\_size=10 Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

| Key         | Value | Description | Bulk Edit |
|-------------|-------|-------------|-----------|
| ✓ page_size | 10    |             |           |
| Key         | Value | Description |           |

Body Cookies 1 Headers 4 Test Results 200 OK • 64 ms • 944 B • Save Response

{ } JSON Preview Visualize

```
1  [
2    {
3      "id": 5,
4      "trialTitle": "Phase II Study #1",
5      "sponsor": "Mass General",
6      "notes": []
7    },
8    {
9      "id": 6,
10     "trialTitle": "Phase II Study #2",
11     "sponsor": "Duke Health",
12     "notes": []
13   },
14   {
15     "id": 7,
16     "trialTitle": "Phase III Study #3",
17     "sponsor": "NYU Langone",
18     "notes": []
19   },
20   {
21     "id": 8,
22     "trialTitle": "Observational Study #4",
23     "sponsor": "UCSF Health",
24     "notes": []
25   },
26   {
27     "id": 9,
28     "trialTitle": "Observational Study #5",
29     "sponsor": "Cleveland Clinic",
30     "notes": []
31   },
32   {
33     "id": 10,
34     "trialTitle": "Phase III Study #6",
35     "sponsor": "Mayo Clinic"
```

⚡ All systems are go Globals Vault Tools

status 200, response body showing 10 trials with **notes** arrays

My Collection > Naive endpoint — 3 requests

SaveShare

GET

http://localhost:8509/api/trials/naive/list?page\_size=50

Send

Docs

Params

Authorization

Headers 7

Body

Scripts

Settings

Cookies

Query Params

| Key       | Value | Description | Bulk Edit |
|-----------|-------|-------------|-----------|
| page_size | 50    |             |           |
| Key       | Value | Description |           |

Body

Cookies 1

Headers 4

Test Results

200 OK • 200 ms • 4.13 KB

Save Response

JSON

Preview

Visualize

```
48     "notes": []
49   },
50   {
51     "id": 13,
52     "trialTitle": "Observational Study #9",
53     "sponsor": "Stanford Medicine",
54     "notes": []
55   },
56   {
57     "id": 14,
58     "trialTitle": "Phase II Study #10",
59     "sponsor": "Cleveland Clinic",
60     "notes": []
61   },
62   {
63     "id": 15,
64     "trialTitle": "Phase III Study #11",
65     "sponsor": "Mass General",
66     "notes": []
67   },
68   {
69     "id": 16,
70     "trialTitle": "Observational Study #12",
71     "sponsor": "Cleveland Clinic",
72     "notes": []
73   },
74   {
75     "id": 17,
76     "trialTitle": "Observational Study #13",
77     "sponsor": "Johns Hopkins",
78     "notes": []
79   },
80   {
81     "id": 18,
82     "trialTitle": "Phase III Study #14",
```

All systems are go

Globals Vault Tools

Search

11:43 PM

The screenshot shows a REST client interface with a collection named "Naive endpoint" containing 3 requests. The selected request is a GET request to `http://localhost:8509/api/trials/naive/list?page_size=200`. The response is a 200 OK status with a response time of 619 ms and a body size of 16.43 KB. The response body is a JSON array of trial objects.

| Key         | Value | Description | Bulk Edit |
|-------------|-------|-------------|-----------|
| ✓ page_size | 200   |             |           |
| Key         | Value | Description |           |

```
{
  "id": 41,
  "trialTitle": "Phase I Study #37",
  "sponsor": "Cleveland Clinic",
  "notes": []
},
{
  "id": 42,
  "trialTitle": "Phase I Study #38",
  "sponsor": "Johns Hopkins",
  "notes": []
},
{
  "id": 43,
  "trialTitle": "Phase I Study #39",
  "sponsor": "Mass General",
  "notes": []
},
{
  "id": 44,
  "trialTitle": "Phase III Study #40",
  "sponsor": "Mayo Clinic",
  "notes": []
},
{
  "id": 45,
  "trialTitle": "Phase I Study #41",
  "sponsor": "Cleveland Clinic",
  "notes": []
},
{
  "id": 46,
  "trialTitle": "Observational Study #42",
  "sponsor": "NYU Langone",
  "notes": []
}
```

## Part 4: Build a Grounded RAG Question-Answering System

### 1) Corpus and Index

The corpus consists of 5 documents written for the clinical trials domain, covering: trial phases (I-IV), informed consent, trial status definitions (matching the same status values used in DOMAIN\_SCHEMA.md), adverse event reporting, and trial registration (NCT numbers). Two documents (trial phases and trial registration) intentionally share overlapping content about trial phases, to create a case where multiple sources contain similar information.

Each document was chunked using `chunk_size = 500` characters and `chunk_overlap = 50` characters, as required. Each chunk stores its text, source document name, and a

unique chunk\_id. The chunks were embedded using the all-MiniLM-L6-v2 sentence-transformer model and loaded into a FAISS vector store (IndexFlatIP, using cosine similarity on normalized vectors).

Running the indexing step produced 28 chunks from 5 documents.

```
models.py U X  measure_hw04.py U X  seed_hw04.py U X  trials.py U X  rag.py U X  database.py U X  main.py M X
ode > rag > rag.py
24 def load_corpus():
26     docs = []
27     for path in sorted(CORPUS_DIR.glob("*.md")):
28         text = path.read_text(encoding="utf-8")
29         docs.append((path.name, text))
30     return docs
31
32
33 def chunk_text(text, chunk_size=CHUNK_SIZE, overlap=CHUNK_OVERLAP):
34     """Character-based chunking with overlap, as required by the assignment."""
35     text = re.sub(r"\s+", " ", text).strip()
36     chunks = []
37     start = 0
38     while start < len(text):
39         end = start + chunk_size
40         chunk = text[start:end]
41         chunks.append(chunk)
42         if end >= len(text):
43             break
44         start = end - overlap
45     return chunks
46
47
48 def build_index():
49     """Chunk every document, embed the chunks, and build a FAISS index."""
50     docs = load_corpus()
51     embedder = get_embedder()
52
53     chunk_records = [] # each: {chunk_id, source, text}
54     chunk_id = 0
55     for source_name, text in docs:
56         pieces = chunk_text(text)
57         for piece in pieces:
58             chunk_records.append({
59                 "chunk_id": chunk_id,
60                 "source": source_name,
61                 "text": piece,
62             })
63             chunk_id += 1
64
65     texts = [c["text"] for c in chunk_records]
66     embeddings = embedder.encode(texts, convert_to_numpy=True, normalize_embeddings=True)
67
68     dim = embeddings.shape[1]
```

**Output:**



The same question was run through three different setups to see how much the context given to the model changes the answer.

Config A : No-RAG. The question is sent straight to the LLM with no document context at all. The model answers only from what it already knows. It gave a correct answer, but it included extra details (like "pharmacokinetics") that aren't in the corpus, and it has no way to show where the answer came from.

Config B: Basic-RAG. The top 3 chunks from the corpus are pasted directly into the prompt, with no cleanup. The model's answer is now grounded in the actual document text and matches it closely, but it still doesn't say which source it used.

Config C: Context-Engineered RAG. Same 3 chunks, but duplicates are removed, each chunk is labeled with its source number, and the model is told to only answer from the sources and to cite the source it used. The answer is almost identical to Config B, but now ends with [\[Source 1\]](#), so the answer can be checked against the real document.

What this shows: No-RAG relies on the model's own memory and adds unverifiable details. Basic-RAG grounds the answer in real text but isn't traceable. Context-RAG keeps the grounding and adds a citation, making it possible to verify the answer came from the actual corpus.

Code Snippet:

```
110     )
111     return response.message.content
112
113
114     def config_a_no_rag(question):
115         """Baseline: question straight to the LLM, no context at all."""
116         prompt = f"Answer this question as best you can:\n\n{question}"
117         return call_llm(prompt)
118
119
120     def config_b_basic_rag(question, index, chunk_records, top_k=3):
121         """Basic RAG: top-k raw chunks pasted into the prompt, no cleanup."""
122         results = retrieve(question, index, chunk_records, top_k=top_k)
123         context = "\n\n".join(r["text"] for r in results)
124         prompt = f"""Use the following context to answer the question.
125
126 Context:
127 {context}
128
129 Question: {question}"""
130         return call_llm(prompt), results
131
132
133     def config_c_context_engineered(question, index, chunk_records, top_k=3):
134         """Context-engineered RAG: dedup, label sources, add grounding rules."""
135         results = retrieve(question, index, chunk_records, top_k=top_k)
136
137         # Drop exact-duplicate chunk text (de-duplication)
138         seen_text = set()
139         deduped = []
140         for r in results:
141             if r["text"] not in seen_text:
142                 seen_text.add(r["text"])
143                 deduped.append(r)
144
145         # Order and label survivors with their source, numbered for citation
146         labeled_context = "\n\n".join(
147             f"[Source {i+1}]: {r['source']}\n\n{r['text']}"
148             for i, r in enumerate(deduped)
149         )
150
151         prompt = f"""Answer the question using ONLY the numbered sources below
```

```

loading weights: 100% | 103/103 [00:00:00:00, 748.87it/s]
Indexed 28 chunks from 5 documents.

=====
CONFIG A: No-RAG
=====
During a Phase I clinical trial, the primary goal is to evaluate the safety, tolerability, and pharmacokinetics (how the body processes the drug) of a new medication in a small group of human volunteers. These trials typically involve 20 to 100 healthy volunteers or patients with the disease the drug is intended to treat.

Key aspects of a Phase I trial include:

- Safety Assessment: Researchers monitor participants for any adverse effects or side effects, determining the maximum tolerated dose and identifying any dose-related risks.
- Pharmacokinetics: Scientists study how the drug is absorbed, distributed, metabolized, and excreted by the body.
- Dosage Finding: The trial helps determine the optimal starting dose and dosing schedule for future phases.
- Limited Efficacy Testing: While safety is the main focus, researchers may gather preliminary data on whether the drug shows any biological activity or potential effectiveness.

Phase I trials are usually the first time a drug is tested in humans and are conducted in a controlled, closely monitored environment. They are essential for ensuring that a drug is safe enough to proceed to later, larger trials involving more participants and broader patient populations.

=====
CONFIG B: Basic-RAG
=====

Query: What happens during a Phase I clinical trial?
Retrieved 3 chunks:
[chunk 0] source=doc1_trial_phases.md score=0.754 "# Clinical Trial Phases Clinical trials that test a new drug or treatment in humans are organized i..."
[chunk 24] source=doc5_trial_registration.md score=0.696 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 2] source=doc1_trial_phases.md score=0.675 "Inue to monitor safety, but now also compare outcomes, sometimes against a placebo or an existing tr..."

Answer: During a Phase I clinical trial, a new treatment is first tested in humans. These trials are small, usually enrolling 20 to 100 healthy volunteers or patients. The main goal is to test safety: researchers look for side effects, determine a safe dosage range, and see how the drug is absorbed and processed by the body.

=====
CONFIG C: Context-Engineered RAG
=====

Query: What happens during a Phase I clinical trial?
Retrieved 3 chunks:
[chunk 0] source=doc1_trial_phases.md score=0.754 "# Clinical Trial Phases Clinical trials that test a new drug or treatment in humans are organized i..."
[chunk 24] source=doc5_trial_registration.md score=0.696 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 2] source=doc1_trial_phases.md score=0.675 "Inue to monitor safety, but now also compare outcomes, sometimes against a placebo or an existing tr..."

Answer: During a Phase I clinical trial, a new treatment is first given to humans. These trials are small, usually enrolling 20 to 100 healthy volunteers or patients. The main goal is to test safety: researchers look for side effects, determine a safe dosage range, and see how the drug is processed by the body [Source 1].
PS D:\data260-3209\code\rag>

```

4)

```

197
198
199 def run_all_configs(question, index, chunk_records, top_k=3):
200     """Run one question through all 3 configs. Returns a dict of results."""
201     answer_a = config_a_no_rag(question)
202     answer_b, chunks_b = config_b_basic_rag(question, index, chunk_records, top_k)
203     answer_c, chunks_c = config_c_context_engineered(question, index, chunk_records, top_k)
204
205     return {
206         "question": question,
207         "config_a_answer": answer_a,
208         "config_b_answer": answer_b,
209         "config_b_chunks": chunks_b,
210         "config_c_answer": answer_c,
211         "config_c_chunks": chunks_c,
212     }
213
214
215 def run_test_set():
216     index, chunk_records = build_index()
217     all_results = {}
218
219     for qid, question in QUESTIONS.items():
220         print("\n" + "#" * 70)
221         print(f"{qid}: {question}")
222         print("#" * 70)
223
224         result = run_all_configs(question, index, chunk_records)
225         all_results[qid] = result
226
227         print("\n--- Config A (No-RAG) ---")
228         print(result["config_a_answer"])
229
230         print("\n--- Config B (Basic-RAG) ---")
231         print_retrieval(question, result["config_b_chunks"])
232         print("Answer:", result["config_b_answer"])
233
234         print("\n--- Config C (Context-RAG) ---")
235         print_retrieval(question, result["config_c_chunks"])
236         print("Answer:", result["config_c_answer"])
237
238     return all_results
239
240

```

Output:

```
PS D:\data260-3209\code\rag> py -3.12 rag.py
Building index from corpus...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%
Indexed 28 chunks from 5 documents.

=====
CONFIG A: No-RAG
=====
During a Phase I clinical trial, the primary goal is to evaluate the safety, tolerability, and pharmacokinetics (how the body processes the drug) of a new medication in a small group of people-typically 20 to 100 healthy volunteers or patients with the disease being treated.

Key aspects of a Phase I trial include:

- **Safety Assessment**: Researchers determine the drug's safety profile, including identifying any side effects and their severity.
- **Dosing Studies**: The trial tests different doses to find the maximum tolerated dose (MTD) and to understand how the body absorbs, distributes, metabolizes, and excretes the drug (pharmacokinetics).
- **Small Sample Size**: Because the focus is on safety, the number of participants is small, usually between 20 and 100 individuals.
- **Volunteers**: Participants are often healthy individuals or patients with a specific condition, depending on the drug being tested.
- **Monitoring**: Close monitoring is conducted to detect adverse effects and ensure participant safety.

Phase I trials are the first step in human testing and are crucial for determining whether a drug is safe enough to proceed to later, larger trials (Phase II and III). These trials do not typically assess the drug's effectiveness or efficacy in treating a disease.

=====
CONFIG B: Basic-RAG
=====
Query: What happens during a Phase I clinical trial?
Retrieved 3 chunks:
[chunk 0] source-docs_trial_phases.md score=0.754 "# Clinical Trial Phases Clinical trials that test a new drug or treatment in humans are organized i..."
[chunk 24] source-docs_trial_registration.md score=0.696 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 2] source-docs_trial_phases.md score=0.675 "line to monitor safety, but now also compare outcomes, sometimes against a placebo or an existing tr..."
Answer: During a Phase I clinical trial, a new treatment is first tested in humans. These trials are small, usually enrolling 20 to 100 healthy volunteers or patients. The main goal is to test safety: researchers look for side effects, determine a safe dosage range, and see how the drug is absorbed and processed by the body.

=====
CONFIG C: Context-Engineered RAG
=====
Query: What happens during a Phase I clinical trial?
Retrieved 3 chunks:
[chunk 0] source-docs_trial_phases.md score=0.754 "# Clinical Trial Phases Clinical trials that test a new drug or treatment in humans are organized i..."
[chunk 24] source-docs_trial_registration.md score=0.696 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 2] source-docs_trial_phases.md score=0.675 "line to monitor safety, but now also compare outcomes, sometimes against a placebo or an existing tr..."
Answer: During a Phase I clinical trial, a new treatment is first given to humans. These trials are small, usually enrolling 20 to 100 healthy volunteers or patients. The main goal is to test safety: researchers look for side effects, determine a safe dosage range, and see how the drug is absorbed and processed by the body [Source 1].
Building index from corpus...

=====
Q1: What is a Phase I clinical trial?
=====
```

```
Problems Output Debug Console Terminal Ports
PS D:\data260-3209\code\rag> py -3.12 rag.py
=====
Q1: What is a Phase I clinical trial?
=====

--- Config A (No-RAG) ---
A Phase I clinical trial is the first stage of human testing for a new drug or medical treatment. It primarily focuses on evaluating the safety, tolerability, and pharmacokinetics (how the body processes the drug) of the treatment in a small group of people-typically between 20 and 100 healthy volunteers or patients with a specific disease.

Key objectives of a Phase I trial include:

- Determining the safe dosage range (the maximum tolerated dose).
- Identifying side effects and adverse reactions.
- Assessing how the drug is absorbed, distributed, metabolized, and excreted by the body.

Phase I trials are usually conducted in a controlled, research setting and are designed to ensure that the treatment is safe before moving on to larger studies involving more patients. These trials do not typically assess the drug's effectiveness in treating a disease, as that is the goal of later phases (Phase II and III).

In summary, a Phase I clinical trial is a critical early step in drug development, aimed at establishing safety and dosing in humans.

--- Config B (Basic-RAG) ---
Query: What is a Phase I clinical trial?
Retrieved 3 chunks:
[chunk 0] source-docs_trial_phases.md score=0.772 "# Clinical Trial Phases Clinical trials that test a new drug or treatment in humans are organized i..."
[chunk 24] source-docs_trial_registration.md score=0.697 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 2] source-docs_trial_phases.md score=0.655 "line to monitor safety, but now also compare outcomes, sometimes against a placebo or an existing tr..."
Answer: A Phase I clinical trial is the first time a new treatment is given to humans. These trials are small, usually enrolling 20 to 100 healthy volunteers or patients. The main goal is to test safety: researchers look for side effects, determine a safe dosage range, and see how the drug is absorbed and processed by the body.

--- Config C (Context-RAG) ---
Query: What is a Phase I clinical trial?
Retrieved 3 chunks:
[chunk 0] source-docs_trial_phases.md score=0.772 "# Clinical Trial Phases Clinical trials that test a new drug or treatment in humans are organized i..."
[chunk 24] source-docs_trial_registration.md score=0.697 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 2] source-docs_trial_phases.md score=0.655 "line to monitor safety, but now also compare outcomes, sometimes against a placebo or an existing tr..."
Answer: A Phase I clinical trial is the first time a new treatment is given to humans. These trials are small, usually enrolling 20 to 100 healthy volunteers or patients. The main goal is to test safety: researchers look for side effects, determine a safe dosage range, and see how the drug is absorbed and metabolized [Source 1].

=====
Q2: What must be included when a clinical trial is registered, and what happens if a trial is stopped early?
=====

--- Config A (No-RAG) ---
When a clinical trial is registered, the following key information must typically be included to ensure transparency, accountability, and scientific integrity:

1. **Trial Title**: A clear and descriptive title that identifies the study.
2. **Study Objective**: The primary and secondary objectives of the trial (e.g., to evaluate efficacy, safety, or both).
3. **Intervention(s)**: A description of the treatments, interventions, or procedures being tested.
4. **Study Design**: Information about the trial design (e.g., randomized, placebo-controlled, double-blind, crossover).
5. **Eligibility Criteria**: The inclusion and exclusion criteria for participants.
```

```
Problems Output Debug Console Terminal Ports
PS D:\data260-3209\code\rag> py -3.12 rag.py
5. **Eligibility Criteria**: The inclusion and exclusion criteria for participants.
6. **Sample Size**: The number of participants planned or enrolled.
7. **Recruitment Plan**: How participants will be recruited and where.
8. **Timeline**: The planned start and end dates, including key milestones.
9. **Contact Information**: The name and contact details of the principal investigator or sponsor.
10. **Trial Protocol**: A detailed description of the trial procedures, including methods of data collection and analysis.
11. **Regulatory Status**: Information about ethical approvals and regulatory submissions.
12. **Sponsor and Investigator Information**: Names and affiliations of the trial sponsors and investigators.
13. **Data Monitoring Plan**: How safety and efficacy will be monitored during the trial.
14. **Trial Registration Number**: A unique identifier assigned by the registry (e.g., ClinicalTrials.gov, WHO ICTRP).

These details are required to ensure that the public and scientific community can access and understand the trial, promote transparency, and prevent fraud or data manipulation.

**What happens if a trial is stopped early?**

If a clinical trial is stopped early—due to safety concerns, lack of efficacy, unexpected results, or other significant reasons—several actions typically occur:

1. **Reason for Early Termination**: The reason must be clearly documented and justified (e.g., futility, serious adverse events, or significant benefit).
2. **Updated Protocol or Report**: The trial protocol is updated or a protocol amendment is issued to reflect the new status.
3. **Data Management and Analysis**: All collected data are preserved and analyzed to understand the outcome of the early termination.
4. **Regulatory and Ethical Review**: The decision is often reviewed by an independent data monitoring committee (DMC) and/or ethics committee to ensure that the stopping was justified and safe.
5. **Public Disclosure**: The trial registry is updated to reflect the early termination, including the reason and outcome. This ensures transparency and allows stakeholders to understand the full trajectory of the trial.
6. **Publication of Results**: The final results, including the early termination and analysis, are typically published in a peer-reviewed journal.
7. **Ongoing Compliance**: The trial must remain compliant with regulatory and ethical standards, and any post-trial activities (e.g., follow-up, data sharing) must be conducted appropriately.

In summary, early termination must be justified, properly documented, and transparently communicated. This helps maintain public trust and ensures that scientific and medical communities can learn from both successful and terminated trials.

--- Config B (Basic-RAG) ---

Query: What must be included when a clinical trial is registered, and what happens if a trial is stopped early?
Retrieved 3 chunks:
[chunk 24] source-docs_trial_registration.md score=0.725 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 14] source-doc3_trial_status.md score=0.664 "nt enrollment, funding problems, or an independent safety board recommending the trial stop because ..."
[chunk 22] source-doc5_trial_registration.md score=0.637 "# Clinical Trial Registration Requirements Before a clinical trial can enroll participants, it gene..."
Answer: When a clinical trial is registered, it must include the following:
- Eligibility criteria, describing who can and cannot join.
- The sponsor or institution responsible for the trial.
- The current recruitment status of the trial.
```

```
Problems Output Debug Console Terminal Ports
PS D:\data260-3209\code\rag> py -3.12 rag.py

--- Config B (Basic-RAG) ---

Query: What must be included when a clinical trial is registered, and what happens if a trial is stopped early?
Retrieved 3 chunks:
[chunk 24] source-doc5_trial_registration.md score=0.725 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 14] source-doc3_trial_status.md score=0.664 "nt enrollment, funding problems, or an independent safety board recommending the trial stop because ..."
[chunk 22] source-doc5_trial_registration.md score=0.637 "# Clinical Trial Registration Requirements Before a clinical trial can enroll participants, it gene..."
Answer: When a clinical trial is registered, it must include the following:
- Eligibility criteria, describing who can and cannot join.
- The sponsor or institution responsible for the trial.
- The current recruitment status of the trial.
- Contact information for the study team.
- The trial's phase, if it involves testing a drug or biologic.

If a trial is stopped early—such as due to an independent safety board recommending a halt because early results show a clear, positive or negative outcome—this can undermine the purpose of clinical trial registration. Early stopping may lead to issues like funding problems or a lack of transparency, and it may result in selective reporting of only favorable results. Therefore, proper registration and monitoring are essential to maintain the integrity and transparency of clinical research.

--- Config C (Context-RAG) ---

Query: What must be included when a clinical trial is registered, and what happens if a trial is stopped early?
Retrieved 3 chunks:
[chunk 24] source-doc5_trial_registration.md score=0.725 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 14] source-doc3_trial_status.md score=0.664 "nt enrollment, funding problems, or an independent safety board recommending the trial stop because ..."
[chunk 22] source-doc5_trial_registration.md score=0.637 "# Clinical Trial Registration Requirements Before a clinical trial can enroll participants, it gene..."
Answer: When a clinical trial is registered, it must include eligibility criteria, the sponsor or institution responsible for the trial, the current recruitment status, contact information for the study team, and the trial's phase if it involves testing a drug or biologic [Source 1]. If a trial is stopped early, it may happen due to issues such as no longer enrollment, funding problems, or an independent safety board recommending the trial stop because early results already show a clear outcome, positive or negative [Source 2].

#####
Q3: What is a clinical trial phase?
#####

--- Config A (No-RAG) ---
A clinical trial phase is a stage in the process of testing a new medical treatment, drug, or device in human participants to evaluate its safety, effectiveness, and optimal dosage. Clinical trials are typically conducted in four distinct phases, each with specific objectives and increasing levels of participant involvement:

1. **Phase 1**:
- Focuses on assessing safety, dosage, and side effects.
- Involves a small group of healthy volunteers or patients (usually 20-100 people).
- Goal: Determine the safe dosage range and identify any adverse effects.

2. **Phase 2**:
- Evaluates effectiveness and further assesses safety.
- Involves a larger group of patients (usually 100-300) who have the condition being treated.
- Goal: Determine if the treatment works for the intended condition and refine the dosage.

3. **Phase 3**:
- Involves large-scale testing to confirm effectiveness, monitor side effects, and compare the treatment to standard therapies.
- Involves several hundred to thousands of participants.
- Goal: Collect sufficient data to support regulatory approval for market use.

4. **Phase 4**:
- Occurs after the treatment is approved and marketed.
```

```

PS D:\data260-3200\code\rags> py -3.12 rag.py
- Goal: Collect sufficient data to support regulatory approval for market use.

4. **Phase 4**:
- Occurs after the treatment is approved and marketed.
- Involves ongoing monitoring of long-term effects and real-world performance.
- Goal: Gather information about rare or long-term side effects and improve use in broader populations.

Each phase builds on the previous one, ensuring that new treatments are both safe and effective before being made available to the public. Regulatory agencies like the FDA (U.S.) or EMA (Europe) oversee these phases and require rigorous data before approving a treatment.

--- Config B (Basic-RAG) ---
Query: What is a clinical trial phase?
Retrieved 3 chunks:
[chunk 0] source-doc1_trial_phases.md score=0.731 "a Clinical Trial Phases Clinical trials that test a new drug or treatment in humans are organized i..."
[chunk 24] source-doc5_trial_registration.md score=0.730 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 11] source-doc3_trial_status.md score=0.645 "a Clinical Trial Status Definitions Every clinical trial listed in a public registry is assigned a ..."
Answer: A clinical trial phase is a sequential stage in the process of testing a new drug or treatment in humans. Each phase answers a specific question and builds on the results of the previous phase. The phases are organized to progressively evaluate the safety, dosage, and effectiveness of the treatment. Phase I trials, for example, are the first time a new treatment is given to humans and focus on testing safety, such as identifying side effects and determining a safe dosage range. Subsequent phases expand on this by evaluating efficacy and comparing the treatment to standard therapies.

--- Config C (Context-RAG) ---
Query: What is a clinical trial phase?
Retrieved 3 chunks:
[chunk 0] source-doc1_trial_phases.md score=0.731 "a Clinical Trial Phases Clinical trials that test a new drug or treatment in humans are organized i..."
[chunk 24] source-doc5_trial_registration.md score=0.730 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 11] source-doc3_trial_status.md score=0.645 "a Clinical Trial Status Definitions Every clinical trial listed in a public registry is assigned a ..."
Answer: A clinical trial phase is a sequential stage in the process of testing a new drug or treatment in humans, where each phase answers a different question and builds on the results of the previous phase [Source 1].

=====
Q4: Why might a trial stop?
=====
--- Config A (No-RAG) ---
A trial might stop for several reasons, depending on the context-whether it's a legal trial, a scientific clinical trial, or a business or project trial. Here are common reasons why a trial might be stopped:

**1. Legal Trial (e.g., criminal or civil court):**
- **Insufficient evidence**: The prosecution or plaintiff may not have enough credible evidence to prove their case beyond a reasonable doubt.
- **Jurisdictional issues**: The case may fall outside the court's authority or legal framework.
- **Judicial dismissal**: A judge may dismiss the case due to procedural errors, lack of standing, or violation of constitutional rights.
- **Settlement**: The parties may agree to resolve the case out of court, ending the trial.
- **Deadlock**: If the jury cannot reach a unanimous decision, the trial may be ended or result in a mistrial.
- **New evidence or developments**: A significant new piece of evidence may require a new trial, or the trial may be halted temporarily.

**2. Clinical Trial (medical research):**
- **Safety concerns**: If adverse effects are found in participants, the trial may be stopped to protect human subjects.
- **Lack of efficacy**: If the treatment shows no benefit compared to a placebo or standard care, the trial may be halted.
- **Statistical significance**: The trial may not show a statistically significant effect, leading to termination.
- **Regulatory intervention**: Regulatory bodies (like the FDA) may require a stop or modification of the trial.
- **Ethical violations**: If participant rights are violated, the trial is stopped.
- **Unmet objectives**: The trial fails to meet its primary or secondary endpoints.

**3. Business or Project Trial:**
- **Failure to meet goals**: The trial may not achieve its intended objectives or deliver expected results.

```

```

Problems Output Debug Console Terminal Ports
PS D:\data260-3200\code\rags> py -3.12 rag.py
- **Ethical violations**: If participant rights are violated, the trial is stopped.
- **Unmet objectives**: The trial fails to meet its primary or secondary endpoints.

**3. Business or Project Trial:**
- **Failure to meet goals**: The trial may not achieve its intended objectives or deliver expected results.
- **Budget constraints**: Financial limitations may force the project to stop.
- **Market changes**: Shifts in market demand or competition can make the trial irrelevant.
- **Internal decisions**: Leadership may decide to abandon the trial due to strategic realignment.
- **Technical issues**: Critical failures in processes or systems may render the trial nonviable.

In all cases, stopping a trial is typically a decision made to protect participants, ensure ethical standards, or make efficient use of resources. The specific reason depends on the nature and purpose of the trial.

--- Config B (Basic-RAG) ---
Query: Why might a trial stop?
Retrieved 3 chunks:
[chunk 14] source-doc3_trial_status.md score=0.615 "nt enrollment, funding problems, or an independent safety board recommending the trial stop because ..."
[chunk 13] source-doc3_trial_status.md score=0.548 "ned activities, including the follow-up period for its last enrolled participant. Results may or may..."
[chunk 24] source-doc5_trial_registration.md score=0.485 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
Answer: A trial might stop due to safety concerns identified during the trial, insufficient enrollment, funding problems, or an independent safety board recommending the trial stop due to early results showing a clear outcome-either positive or negative.

--- Config C (Context-RAG) ---
Query: Why might a trial stop?
Retrieved 3 chunks:
[chunk 14] source-doc3_trial_status.md score=0.615 "nt enrollment, funding problems, or an independent safety board recommending the trial stop because ..."
[chunk 13] source-doc3_trial_status.md score=0.548 "ned activities, including the follow-up period for its last enrolled participant. Results may or may..."
[chunk 24] source-doc5_trial_registration.md score=0.485 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
Answer: A trial might stop due to safety concerns identified during the trial, insufficient enrollment, funding problems, or an independent safety board recommending the trial stop because early results already show a clear outcome, positive or negative [Source 2].

=====
Q5: What is the average cost of running a Phase III clinical trial?
=====
--- Config A (No-RAG) ---
The average cost of running a Phase III clinical trial can vary significantly depending on the disease area, trial design, patient population, geography, and the pharmaceutical or biotech company conducting the trial. However, based on industry data and reports from sources such as the Tufts Center for the Study of Drug Development, the U.S. Food and Drug Administration (FDA), and academic research:

**The average cost of a Phase III clinical trial is typically between $100 million and $200 million.**

Some key points:
- **Lower-end estimates** (around $100 million) may apply to well-established drug classes or diseases with relatively small, manageable patient populations.
- **Higher-end estimates** (up to $300 million or more) are common for complex diseases (e.g., rare genetic disorders, oncology, or immunology), trials involving global recruitment, long durations, or novel technologies.
- These costs include expenses for patient recruitment, monitoring, data management, regulatory compliance, site operations, statistical analysis, and safety monitoring.
- The cost is often higher in global trials due to international site management, currency conversion, and regulatory differences.

In summary, while there is no single fixed number, **a reasonable average estimate is approximately $150 million** for a typical Phase III clinical trial in the pharmaceutical industry.

--- Config B (Basic-RAG) ---
Query: What is the average cost of running a Phase III clinical trial?

```

```

PS D:\data260-3209\code\rag> py -3.12 rag.py

In summary, while there is no single fixed number, **a reasonable average estimate is approximately $150 million** for a typical Phase III clinical trial in the pharmaceutical industry.

--- Config B (Basic-RAG) ---

Query: What is the average cost of running a Phase III clinical trial?
Retrieved 3 chunks:
[chunk 2] source-doc1_trial_phases.md score=0.549 "inue to monitor safety, but now also compare outcomes, sometimes against a placebo or an existing tr..."
[chunk 3] source-doc1_trial_phases.md score=0.523 "t (or a placebo, when no standard exists). Phase III is the last major step before a regulatory agen..."
[chunk 0] source-doc1_trial_phases.md score=0.512 "a Clinical Trial Phases Clinical trials that test a new drug or treatment in humans are organized i..."
Answer: The context provided does not specify the average cost of running a Phase III clinical trial. While it mentions that Phase III trials "cost the most to run," it does not give a numerical figure for the average cost. Therefore, based on the given information, the average cost cannot be determined.

--- Config C (Context-RAG) ---

Query: What is the average cost of running a Phase III clinical trial?
Retrieved 3 chunks:
[chunk 2] source-doc1_trial_phases.md score=0.549 "inue to monitor safety, but now also compare outcomes, sometimes against a placebo or an existing tr..."
[chunk 3] source-doc1_trial_phases.md score=0.523 "t (or a placebo, when no standard exists). Phase III is the last major step before a regulatory agen..."
[chunk 0] source-doc1_trial_phases.md score=0.512 "a Clinical Trial Phases Clinical trials that test a new drug or treatment in humans are organized i..."
Answer: I cannot answer this question from the provided documents

=====
Q6: What is the capital of France?
=====

--- Config A (No-RAG) ---
The capital of France is Paris.

--- Config B (Basic-RAG) ---

Query: What is the capital of France?
Retrieved 3 chunks:
[chunk 5] source-doc1_trial_phases.md score=0.093 "ly in its public registration record, alongside its recruitment status..."
[chunk 0] source-doc1_trial_phases.md score=0.045 "a Clinical Trial Phases Clinical trials that test a new drug or treatment in humans are organized i..."
[chunk 23] source-doc5_trial_registration.md score=0.032 "dentifier called an NCT number (for example, NCT01234567). This number is used to reference the tria..."
Answer: The capital of France is Paris.

--- Config C (Context-RAG) ---

Query: What is the capital of France?
Retrieved 3 chunks:
[chunk 5] source-doc1_trial_phases.md score=0.093 "ly in its public registration record, alongside its recruitment status..."
[chunk 0] source-doc1_trial_phases.md score=0.045 "a Clinical Trial Phases Clinical trials that test a new drug or treatment in humans are organized i..."
[chunk 23] source-doc5_trial_registration.md score=0.032 "dentifier called an NCT number (for example, NCT01234567). This number is used to reference the tria..."
Answer: I cannot answer this question from the provided documents

All 6 questions completed.
PS D:\data260-3209\code\rag> █

```

All 6 required question types were tested across the three configurations. Q1 (single-chunk) and Q3 (overlapping info across doc1 and doc5) retrieved the expected chunks and produced consistent answers across configs. Q2 (needing two chunks) was correctly answered by Config C citing both Source 1 and Source 2. Q5 (not in the documents) and Q6 (unrelated) were correctly refused by Config C using the exact required phrase, "I cannot answer this question from the provided documents." Config A and Config B, which have no grounding rule, both answered Q6 anyway (hallucinating from general knowledge, since "Paris" happens to be true but is not grounded in the corpus), and Config B partially avoided answering Q5 but did not use a formal refusal. This shows that the grounding rule added in Config C is what actually enforces correct refusal behavior means retrieval and context alone are not enough.

5)

Code Snippet:

```

def run_k_sweep(question, index, chunk_records, k_values=(1, 3, 5)):
    """Run one question at different top_k values to see how context size affects the answer."""
    sweep_results = {}
    for k in k_values:
        print("\n" + "-" * 60)
        print(f"top_k = {k}")
        print("-" * 60)
        answer, chunks = config_c_context_engineered(question, index, chunk_records, top_k=k)
        print_retrieval(question, chunks)
        print("Answer:", answer)
        sweep_results[k] = {"answer": answer, "chunks": chunks}
    return sweep_results

if __name__ == "__main__":
    print("Building index from corpus...")
    corpus = Corpus.from_instances(

```

## Output snippet:

```

=====
TOP_K SWEEP (Config C) on Q2
=====

-----
top_k = 1
-----

Query: What must be included when a clinical trial is registered, and what happens if a trial is stopped early?
Retrieved 1 chunks:
[chunk 24] source=doc5_trial_registration.md score=0.725 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
Answer: What must be included when a clinical trial is registered: eligibility criteria, the sponsor or institution responsible for the trial, the current recruitment status of the trial, contact information for the study team, and the trial's phase if it involves testing a drug or biologic [Source 1].

The provided sources do not contain information about what happens if a trial is stopped early.

I cannot answer this question from the provided documents

-----
top_k = 3
-----

Query: What must be included when a clinical trial is registered, and what happens if a trial is stopped early?
Retrieved 3 chunks:
[chunk 24] source=doc5_trial_registration.md score=0.725 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 14] source=doc3_trial_status.md score=0.664 "nt enrollment, funding problems, or an independent safety board recommending the trial stop because ..."
[chunk 22] source=doc5_trial_registration.md score=0.637 "# Clinical Trial Registration Requirements Before a clinical trial can enroll participants, it gene..."
Answer: When a clinical trial is registered, it must include eligibility criteria, the sponsor or institution responsible for the trial, the current recruitment status, contact information for the study team, and the trial's phase (if it involves testing a drug or biologic) [Source 1]. If a trial is stopped early, it may happen due to issues such as no longer enrollment, funding problems, or an independent safety board recommending the trial stop because early results already show a clear outcome, positive or negative [Source 2].

-----
top_k = 5
-----

Query: What must be included when a clinical trial is registered, and what happens if a trial is stopped early?
Retrieved 5 chunks:
[chunk 24] source=doc5_trial_registration.md score=0.725 "ment). - Eligibility criteria, describing who can and cannot join. - The sponsor or institution resp..."
[chunk 14] source=doc3_trial_status.md score=0.664 "nt enrollment, funding problems, or an independent safety board recommending the trial stop because ..."
[chunk 22] source=doc5_trial_registration.md score=0.637 "# Clinical Trial Registration Requirements Before a clinical trial can enroll participants, it gene..."
[chunk 11] source=doc3_trial_status.md score=0.634 "# Clinical Trial Status Definitions Every clinical trial listed in a public registry is assigned a ..."
[chunk 12] source=doc3_trial_status.md score=0.626 "stage where interested patients should contact the study team if they want to take part. ## Active, ..."
Answer: When a clinical trial is registered, it must include eligibility criteria, the sponsor or institution responsible for the trial, the current recruitment status of the trial, contact information for the study team, and the trial's phase if it involves testing a drug or biologic [Source 1]. If a trial is stopped early, it may happen due to enrollment problems, funding issues, or an independent safety board recommending the trial stop because early results show a clear outcome, either positive or negative [Source 2].

```

At  $k=1$ , only one relevant chunk was retrieved, so the model could answer the registration half of the question but correctly refused the early-termination half, since no relevant chunk was available. This shows retrieving too little context causes a genuine information gap, not a hallucination — a good sign that the grounding rule is working as intended.

At k=3, the second most relevant chunk (about trial termination) was retrieved alongside the registration chunk, allowing the model to answer both halves correctly, each properly cited.

At k=5, two additional, less relevant chunks were retrieved (about "Recruiting" and "Active, Not Recruiting" status definitions), which were unrelated to this specific question. The answer at k=5 was identical to k=3, meaning the extra context did not help, but did not hurt either. The model correctly ignored the irrelevant chunks rather than incorporating unrelated details.

Best k for this question: k=3. k=1 was too small and caused a partial refusal on an answerable question. k=5 retrieved unnecessary chunks without adding value, and in general, adding irrelevant chunks raises the risk of the model incorporating unrelated information into its answer, especially on harder or more ambiguous questions. This demonstrates that more context is not always better context the right amount of relevant context matters more than simply maximizing k.

6)

| Question | Config          | Correct Retrieval | Correct Answer | Grounded | Refused-When-Needed |
|----------|-----------------|-------------------|----------------|----------|---------------------|
| Q1       | A (No-RAG)      | N/A               | Yes            | No       | N/A                 |
| Q1       | B (Basic-RAG)   | Yes               | Yes            | Yes      | N/A                 |
| Q1       | C (Context-RAG) | Yes               | Yes            | Yes      | N/A                 |
| Q2       | A (No-RAG)      | N/A               | Partial        | No       | N/A                 |
| Q2       | B (Basic-RAG)   | Yes               | Yes            | Yes      | N/A                 |
| Q2       | C (Context-RAG) | Yes               | Yes            | Yes      | N/A                 |
| Q3       | A (No-RAG)      | N/A               | Yes            | No       | N/A                 |
| Q3       | B (Basic-RAG)   | Yes               | Yes            | Yes      | N/A                 |
| Q3       | C (Context-RAG) | Yes               | Yes            | Yes      | N/A                 |
| Q4       | A (No-RAG)      | N/A               | Partial        | No       | N/A                 |
| Q4       | B (Basic-RAG)   | Yes               | Yes            | Yes      | N/A                 |
| Q4       | C (Context-RAG) | Yes               | Yes            | Yes      | N/A                 |
| Q5       | A (No-RAG)      | N/A               | No             | No       | No                  |
| Q5       | B (Basic-RAG)   | Yes               | Yes            | Yes      | Partial             |

|    |                 |     |     |     |     |
|----|-----------------|-----|-----|-----|-----|
| Q5 | C (Context-RAG) | Yes | Yes | Yes | Yes |
| Q6 | A (No-RAG)      | N/A | Yes | No  | No  |
| Q6 | B (Basic-RAG)   | Yes | No  | No  | No  |
| Q6 | C (Context-RAG) | Yes | Yes | Yes | Yes |

### Summary metrics

| Config          | Accuracy                     | Faithfulness (Grounded) | Format Compliance   | Robustness (Q5/Q6) |
|-----------------|------------------------------|-------------------------|---------------------|--------------------|
| A - No-RAG      | 4/6 correct (2 hallucinated) | 0/6 grounded            | N/A                 | 0/2 refused        |
| B - Basic-RAG   | 5/6 correct                  | 5/6 grounded            | No citations used   | 1/2 refused        |
| C - Context-RAG | 6/6 correct                  | 6/6 grounded            | 6/6 followed format | 2/2 refused        |

## 7) Analysis

Across all six test questions, retrieval quality was strong: the FAISS index consistently surfaced the correct chunk for straightforward questions. Q1 retrieved the exact opening chunk of the Phase I section with the highest similarity score (0.754-0.772). Q3 correctly surfaced overlapping content from two different documents (doc1\_trial\_phases.md and doc5\_trial\_registration.md), confirming the corpus's intentional overlap worked as designed to test cross-document consistency. Q2, which needed two separate facts, retrieved both the registration chunk and the termination chunk within the top three results, letting Context-RAG answer both halves correctly with separate citations.

The context changes that helped most were de-duplication and source labeling with grounding rules, not top\_k itself. Comparing Config B and Config C on the same retrieved chunks shows nearly identical answer content, but only Config C reliably refuses out-of-scope questions and cites its sources. This means the grounding rule instruction mattered more than the retrieval step for those specific behaviors. The k-sweep on Q2 showed that top\_k also matters, but has a ceiling: k=1 caused a correct partial refusal due to missing information, k=3 fully answered the question, and k=5 added two irrelevant chunks that did not change the answer but added unnecessary noise. This confirms more context is not always better; the goal is retrieving enough relevant chunks, not the most possible chunks.

On Q5 and Q6, the three configurations diverged sharply. Config A (No-RAG) hallucinated on both: it invented a specific, plausible-sounding but entirely fabricated statistic ("\$150 million") for Q5, and it correctly answered Q6 ("Paris") only because that fact happens to also be true general knowledge, not because it used the corpus. Config B avoided fabricating an answer for Q5 by noting the context lacked a cost figure, but it did not use the required refusal phrase, and on Q6 it

answered directly despite retrieving chunks with near-zero similarity scores (0.032-0.093), meaning it should have recognized the retrieved context was irrelevant but did not. Only Config C reliably refused both questions using the exact required phrase, because its prompt explicitly instructed it to check whether the sources were sufficient before answering.

Overall, retrieval quality determined *what information was available*, but the prompt's grounding rules determined *whether the model used that availability responsibly*. Good retrieval without grounding instructions (Config B) still risks hallucination on unanswerable questions, since the model defaults to being "helpful" rather than accurate. The combination of relevant retrieval, de-duplicated and clearly labeled context, and an explicit instruction to refuse when evidence is insufficient (Config C) was what actually produced accurate, faithful, and appropriately cautious answers across all six question types.

AI USE:

# AI\_USE.md

### ## 1. What I used an AI assistant for, and what I did myself

I used Claude to help write the backend code (FastAPI routes, database models, login sessions), the React frontend, the script that measures the N+1 problem, and the RAG system (splitting documents into chunks, searching them, and writing the prompts for the three setups). Claude also helped me fix bugs as they came up — a duplicate class in models.py, a routing bug that caused a 405 error, an Ollama memory crash, and a small React syntax error.

I did all the actual work myself. I ran every command, installed the packages set up MySQL, tested every API in Postman and in the browser, filled the database with test data, ran the measurement and RAG scripts, and checked the real output at every step before moving on.

### ## 2. One AI-produced output that was wrong or unsuitable, or one thing independently verified

Claude first told me to put two debug routes (/api/debug/query-count and /api/debug/reset-query-count) after the line app.mount("/", StaticFiles(...)) in main.py. This broke the reset route — it gave a 405 Method Not Allowed error, because the static file mount grabbed the request before it could reach my debug routes.

### ## 3. How I detected the problem or verified the result

I tested the reset route in FastAPI's `/docs` page and got a 405 error instead of the 200 I expected. I sent the error and my full `main.py` file back to Claude, and it compared the order of my routes to how FastAPI checks them (top to bottom) and found that the static file mount was catching the request first.

### ## 4. What I changed and why it works now

I moved the two debug routes above the `app.mount(...)` line. FastAPI checks routes in the order they're written, so putting the debug routes first means they get matched before the static file mount can grab the request. After moving them, both debug routes worked and returned 200, which I checked again in PowerShell. .